



A framework for user centred privacy and security in the cloud

Requirements Specification V2

Type (distribution level)	Public
Contractual date of Delivery	31-08-2016
Actual date of delivery	31-08-2016 (Updated on 25/01/2017)
Deliverable number	D2.3
Deliverable name	Requirements Specification V2
Version	1.4
Number of pages	99
WP/Task related to the deliverable	Task 2.2 – User Requirements
WP/Task responsible	AKKA
Author(s)	AKKA (Frédéric Brouillé, Thierry Chevallier); OFFIS (Stefan Janecek); URV (David Sanchez, Alberto Blanco); THALES (Romain Ferrari); Montimage (Antonio M.Ortiz); KUL (Stephanie Mihail)
Partner(s) Contributing	All
Document ID	CLARUS-D2.3-Requirements Specification V2
Abstract	<i>This document is an updated version of the CLARUS functional and technical requirements V1. It contains several modifications and enhancements compared to the previous deliverable version, notably regarding the use cases and functional requirements.</i>

Disclaimer

CLARUS (G.A. 644024) is a Research and Innovation Actions project funded by the EU Framework Programme for Research and Innovation Horizon 2020. This document contains information on CLARUS core activities, findings and outcomes. Any reference to content in this document should clearly indicate the authors, source, organisation and publication date. The content of this publication is the sole responsibility of the CLARUS consortium and cannot be considered to reflect the views of the European Commission.

Revision History

Version	Date	Author	Description
0.1	04/07/2016	AKKA	proposed ToC
0.2	11/07/2016	AKKA	Introduction Refined use cases
0.3	12/07/2016	URV, OFFIS	Intermediate project results WP3 and WP4
0.4	18/07/2016	AKKA	End-user perspective (EUP) and Security Policy (SEC) use cases revamping
0.5	22/07/2016	AKKA	Update of requirements V1
0.6	27/07/2016	AKKA, OFFIS	UML diagrams for the use cases Intermediate project results WP5 Use cases for external proxy registration
0.7	28/07/2016	AKKA, THALES	Intermediate project results WP6 Conclusion
1.0	29/07/2016	AKKA	Version for partner review
1.1	22/08/2016	URV, KUL, AKKA	Integration of partner review
1.2	31/08/2016	AKKA	Deliverable version
1.3	31/08/2016	URV	Format changes
1.4	23/01/2017	AKKA	Added developers' requirements (review comment #7) Resubmission version

Table of Contents

1	INTRODUCTION	5
1.1	THE NEED FOR A CLARUS REQUIREMENTS V2	5
1.2	METHODOLOGY	5
1.3	APPLICABLE AND REFERENCE DOCUMENTS.....	6
1.4	NOTATIONS, ABBREVIATIONS AND ACRONYMS	7
2	INTERMEDIATE PROJECT RESULTS	8
2.1	WORK PACKAGES.....	8
2.2	IMPLICATIONS REGARDING THE INITIAL REQUIREMENTS (V1)	16
3	REFINED USE CASES	19
3.1	REMINDERS	19
3.2	END-USER PERSPECTIVE	21
3.3	SECURITY MANAGEMENT	37
3.4	MONITORING	48
3.5	ADMINISTRATION	54
3.6	DEVELOPMENT.....	61
4	REFINED REQUIREMENTS.....	66
4.1	REMINDERS	66
4.2	FUNCTIONAL REQUIREMENTS.....	68
4.3	NON FUNCTIONAL REQUIREMENTS	87
5	CONCLUSION	99

1 Introduction

1.1 The need for a CLARUS requirements V2

This document is an updated version of the CLARUS functional and technical requirements deliverable. Because the first version was elaborated early in the project, there was the need for a second version that takes into consideration the intermediate project results achieved in the other work packages, e.g. research on new security techniques, architecture considerations and implementation choices. Actually some requirements were no longer applicable while some others needed to be added, and yet others modified. Moreover, this deliverable considers the project reviewers' comments regarding the need for specifying CLARUS developers' requirements.

As described in the first version of this document, CLARUS can be viewed as a proxy running on trusted local premises transparently for the end-user (and, in some cases, as a software to be integrated in the cloud platform).

More precisely, the CLARUS solution is :

- an “enhanced-security solution”
- configurable by a dedicated actor (embodied by the IT security manager of the organization considered),
- able to support multi usage scenarios for outsourcing data to the cloud,
- able to define security policies according to data usage and data attributes regarding confidentiality and privacy,
- encompassing monitoring functionalities, able to face potential intrusion attempt in an ‘Attack Tolerant System’ mode, which delivers appropriated counter-measures, depending on the intrusion type encountered,
- easy to integrate within the IT system of an organization, including access control management capabilities.

1.2 Methodology

1.2.1 Refinement of D2.2 requirements

In order to specify the first version of the functional and non-functional requirements, we conceived the possible uses of the CLARUS solution, and then described the main scenarios corresponding to these uses – keeping in mind the two specific domains of health IS and Geographic IS considered in the project proposal. The requirements of the CLARUS solution largely depend on these scenarios. Any evolution/change/modification in them impacts the requirements for CLARUS.

A year has gone since the initial definition of the CLARUS requirements v1, and our common approach to CLARUS has been refined. From a broad picture of the CLARUS solution at the initial stage of the project, the consortium progressively reached a common detailed vision of what is/should be the CLARUS platform.

1.2.2 Inclusion of intermediate project results

The structure of the document is quite similar to the one of requirements V1, i.e. the document describes the main use cases of the solution and infers the corresponding requirements. However an additional section has been introduced, namely the “*intermediate project results*”. It was indeed important to take into consideration the inputs from other work packages (such as the research results on security-enabling techniques, the consensued architecture or the envisioned exploitation plan) as they may have an impact on how the use cases would be enforced in practice and, thus, on the requirements as they were initially expressed.

1.2.3 Inclusion of Interim Review recommendations

In addition, following the Interim Review of the project according to which "it is recommended to extend the work on requirements to represent the developers' requirements" (recommendation No.7), we have :

- refined the actors, stressing out the difference between the Developer and the Administrator (or IT team)
- refined the use cases, by adding a comprehensive set of use cases centered around the Developer
- refined the requirements, by adding specific Non-Functional Requirements specifically dedicated to Development

Other recommendations regarding e.g. legal requirements related to governmental users are covered in the D2.4 [R1].

1.3 Applicable and Reference documents

App. /Ref.	Title
[A1]	CLARUS Grant Agreement 644024
[A2]	CLARUS 644024 Consortium Agreement v3.0, dec.2014
[A3]	CLARUS Glossary
[A4]	CLARUS D2.1 – Definition of Application Cases
[A5]	CLARUS D2.2 – Requirements Specification V1
[R1]	CLARUS D2.4 – Legal and Ethical Requirements
[R2]	CLARUS D2.5 – Standardisation Requirements
[R3]	Directive 95/46/EC of the European Parliament and of the Council of 24/10/1995 on the protection of individuals with regard to the processing of personal data and on the free movement of such data (Data Protection Directive), O.J. L 281, http://eurlex.europa.eu/LexUriServ/LexUriServ.do?uri=CELEX:31995L0046:en:HTML

1.4 Notations, Abbreviations and Acronyms

<i>ADM</i>	Use case classification code for 'Administration'
<i>API</i>	Application Programming Interface
<i>CH</i>	Requirement priority for 'Could Have'
<i>CSP</i>	Cloud Service Provider
<i>EUP</i>	Use case classification code for 'End-User Perspective'
<i>GA</i>	Grant Agreement
<i>IaaS</i>	Infrastructure as a Service
<i>IMPL</i>	Use case classification code for 'Design and Implementation'
<i>IS</i>	Information System
<i>IT</i>	Information Technology
<i>JDBC</i>	Java DataBase Connectivity
<i>LDAP</i>	Lightweight Directory Access Protocol
<i>MAMD</i>	Monitoring and Attack Management Dashboard
<i>MH</i>	Requirement priority for 'Must Have'
<i>MON</i>	Use case classification code for 'Monitoring'
<i>OS</i>	Operating System
<i>PaaS</i>	Platform as a Service
<i>REQ</i>	Requirement naming scheme prefix for 'Requirement'
<i>REST</i>	Representational State Transfer
<i>S3</i>	Simple Storage Service
<i>SaaS</i>	Software as a Service
<i>SEC</i>	Use case classification code for 'Security Management'
<i>SH</i>	Requirement priority for 'Should Have'
<i>SQL</i>	Structured Query Language
<i>SSO</i>	Single Sign-On
<i>UC</i>	Use Case
<i>UML</i>	Unified Modeling Language
<i>URI</i>	Uniform Resource Identifier
<i>WL</i>	Requirement priority for 'would like'
<i>WP</i>	Work Package
<i>WSDL</i>	Web Services Description Language
<i>XML</i>	eXtensible Markup Language

2 Intermediate project results

2.1 Work packages

The aim of this section is to collect, for each of the CLARUS work packages, all the elements that appear to have an impact on the user and technical requirements of the solution.

2.1.1 Research on security and Privacy (WP3)

This section summarizes the research results on security and privacy-preserving methods developed so far in WP3. For each method, we highlight the supported functionalities, the performance impact (if any) they may cause on the CLARUS/CSP sides, the data utility that users and CSPs retain on the data and any other functional or technical constraints that could be considered in the design and deployment of the system.

1.1.1.1 Definitions

For coherence with the terminology used in D5.1, we define the following modes regarding the way in which data are handled by the modules:

- *monolithic* if the operation requires transforming the dataset as a whole;
- *one at a time* if the operation transforms the records one by one (by means e.g. of a unique identifier) and independently of each other;
- *as it is* if the operation does not transform the data;
- *orchestrated* if the operation replaces the original request with a set of orchestrated requests.

1.1.1.2 Data anonymization

Data anonymization irreversibly transforms data in a privacy preserving way. The outcome is still clear data that can be of use for the CSPs and external users, but with a lower accuracy (and, thus, a lower disclosure risk) than the original data. Data anonymization is performed once at the storage stage; after that, any query on the data (search, retrieval, calculations) are transparent for CLARUS and the CSP, even though they may result in approximate results.

Two types of anonymization mechanisms have been designed:

- *Data coarsening*: it systematically generalizes input records (independently, *one at a time*) according to a user-defined coarsening level. Because coarsened data is less detailed than the original one, disclosure risk is minimized.
- *Data microaggregation*: it clusters a fixed number of similar records together and replace them with average values; thus, it transforms the whole dataset in a *monolithic* way. Because microaggregated records are indistinguishable according to the cluster cardinality, the re-identification probability is lowered.

Supported functionalities and corresponding constraints

- *Storage*
 - for microaggregation (the whole dataset is anonymized as a *monolithic* block)
 - for coarsening (records are generalized *one at a time*)
- *Update* is only supported for data coarsening (records are generalized *one at a time*)
- *Retrieval* (request forwarded *as-it-is* to the CSP)

- *Search* (request forwarded *as-it-is* to the CSP)
- *Computation* (request is forwarded *as-it-is* to the CSP)

Also, because search/retrieval/computation operations are fully transparent (i.e., no CLARUS transformations are needed to get the results), they can be offered to external users not using CLARUS.

Impacts on efficiency

Data coarsening incurs a linear cost on local premises (with regard to the size of the dataset) at the storage phase to generalize data. Data microaggregation incurs a quadratic cost on local premises (according to the size of the dataset) at the storage phase to microaggregate data. Both approaches are fully transparent for search/retrieval/computation, with no additional computational burdens for neither the client, CLARUS or the CSP.

Impacts on data utility

Because anonymized data cannot be de-anonymized, *partial* utility is preserved both for the CSP and the end-user (or external users not using CLARUS). Because microaggregated records are indistinguishable according to the cluster cardinality, updates are not supported.

1.1.1.3 Data splitting

Data splitting makes a local partition of the sensitive data and separately stores data fragments in different CSPs, in a way that each individual fragment does not cause privacy risks; data fragments are stored in clear and are unmodified so that functionalities and data accuracy are efficiently preserved. Because data is split and only CLARUS knows the exact cloud locations for a given dataset, CLARUS performs an adaptation (and/or orchestration) of user queries and an aggregation of the results.

Supported functionalities and corresponding constraints

- *Storage* (records of the dataset are split *one at a time*)
- *Update* (records to update are split *one at a time*)
- *Retrieval* (results are built *one at a time* aggregating from partial data stored in different CSP)
- *Search* (results are built *one at a time* aggregating from partial data stored in different CSP)
- *Computation* (*orchestrated*, replacing the whole computation process running on the cloud by a set of partial calculations performed on the different clouds)

Impacts on efficiency

Data splitting incurs a linear cost on local premises (with regard to the size of the dataset) to split records. The adaptation of search/retrieval queries and the posterior aggregation scales linearly according to the number of data fragments; in any case, split queries can be parallelized to the different CSPs storing partial data. The cost of computations depend on the complexity of the calculus and the orchestration needed to implement the operation and may introduce burdens both for the local CLARUS proxy and for the CSP.

Impacts on data utility

The end-user retains full utility on the data and the results search, retrieval and computations. Each CSP storing data fragment retains full utility on the fragment it stores (which is a part of the utility of the whole dataset).

1.1.1.4 Encryption

Data encryption is a method to protect data in a secure and reversible way. The scheme requires a secret key that is used both to encrypt and to decrypt data: it is a symmetric-key encryption scheme. The security of the scheme is based on the hardness of the best algorithm known to recover the original data, in the case a user does not have the secret key. The encrypted data can only be accessed by the users owning the key. Encryption is performed by CLARUS once at the storage stage, and the decryption is performed after recovering the encrypted data from the CSP. The keys are stored at the proxy. In this way, the CSP never has access to the plaintext data or to the keys.

Supported functionalities and corresponding constraints

- *Storage* (records of the dataset are encrypted *one at a time*)
- *Update* (records to update are encrypted *one at a time*)
- *Retrieval* (data are retrieved *one at a time* and decrypted on CLARUS premises)

Impacts on efficiency

Encryption incurs a linear cost on local premises (with regard to the size of the dataset) to encrypt the data. Current symmetric-key encryption schemes like AES can be implemented efficiently. Decryption is also efficient.

Impacts on data utility

Because the purpose of this method is just to store data in a secure way, it is not possible to make computations on the encrypted data and searches by attribute values are also not supported. The CSP does not retain any utility on the encrypted data it stores, but data retrieval produces perfectly accurate results for end users.

1.1.1.5 Homomorphic encryption

Homomorphic encryption is used to store data in a secure way that allows to perform certain computations on the encrypted data. The encryption is reversible. The security of the scheme is based on the hardness of the mathematical problem that is needed to solve to recover the original data. The encryption scheme is a public-key encryption scheme, and the secret key is stored in CLARUS. The secret key is used to decrypt data. The encrypted data can only be decrypted by the users owning the secret key. Encryption is performed by CLARUS once at the storage stage, and the decryption is performed after recovering the encrypted data from the CSP. Before recovering the data, it is possible to perform sums on the encrypted data. In this way, the user can recover a value that is the output of a linear function.

Supported functionalities and corresponding constraints

- *Storage* (records of the dataset are encrypted *one at a time*)
- *Update* (records to update are encrypted *one at a time*)
- *Retrieval* (data retrieved *one at a time* and decrypted on CLARUS premises)
- *Computation* (*orchestrated*, replacing the whole computation process running on the cloud by a set of partial calculations performed on the cloud and on premises)

Impacts on efficiency

Encryption incurs a linear cost on local premises (with regard to the size of the dataset) to encrypt the data. However, additively-homomorphic encryption schemes like the Pallier scheme is much more expensive than the common symmetric-key encryption schemes like AES.

Impacts on data utility

The main difference between using plain symmetric-key encryption schemes and homomorphic encryption schemes is that using the latter it is possible for the end user to request computations (with perfectly accurate results) on encrypted outsourced data. As above, the CSP retains no utility at all on the encrypted data it stores.

1.1.1.6 Searchable encryption

Searchable encryption is used to store data in a secure way that allows to perform queries on the encrypted data. The encryption is reversible. The security of the scheme is based on the hardness of the mathematical problem that is needed to recover the original data. The scheme is a symmetric encryption scheme and requires a secure key that is used both to encrypt and to decrypt data. The encrypted data can only be accessed by the users owning the key. Encryption is performed by CLARUS once at the storage stage, and the decryption is performed after recovering the encrypted data from the CSP. The user can perform queries on the encrypted data. The answer of the query is a link to the encrypted document containing the keywords in the query.

Supported functionalities and corresponding constraints

- *Storage* (creates a search index over the whole dataset, processed as a *monolithic* block)
- *Retrieval* (data is retrieved *one at a time* and decrypted on CLARUS premises)
- *Search* (results are decrypted *one at a time*)

Impacts on efficiency

Encryption using searchable encryption involves the creation of an encrypted index that will be used later to find the files including certain keywords. The creation of this encrypted index is the operation that is harder to compute. The queries to the encrypted index are reasonably fast, but the complexity of the queries revert on longer messages sent to the CSP and more expensive executions. The use of searchable encryption includes also the use of symmetric-key encryption schemes to store the indexed data in a secure way. The efficiency of this scheme is described above.

Moreover, because searchable encryption relies on a search index over the whole dataset, updates of a dataset imply to rebuild the search index. This operation comes at a cost of retrieving and decrypting the whole dataset on premises.

Impacts on data utility

The main difference between using plain symmetric-key encryption schemes and searchable encryption schemes is that using the later it is possible to make queries (with perfectly accurate results) on the encrypted data. When data is encrypted, the owner of the data selects some keywords that are considered to be useful in the future. Then the index is created according to these keywords. Any search query asking for some of these keywords will be possible in the future. As above, the CSP retains no utility at all on the encrypted data it stores. Moreover, updates come at a cost and therefore can be considered as unsupported.

2.1.2 Architecture design (WP4)

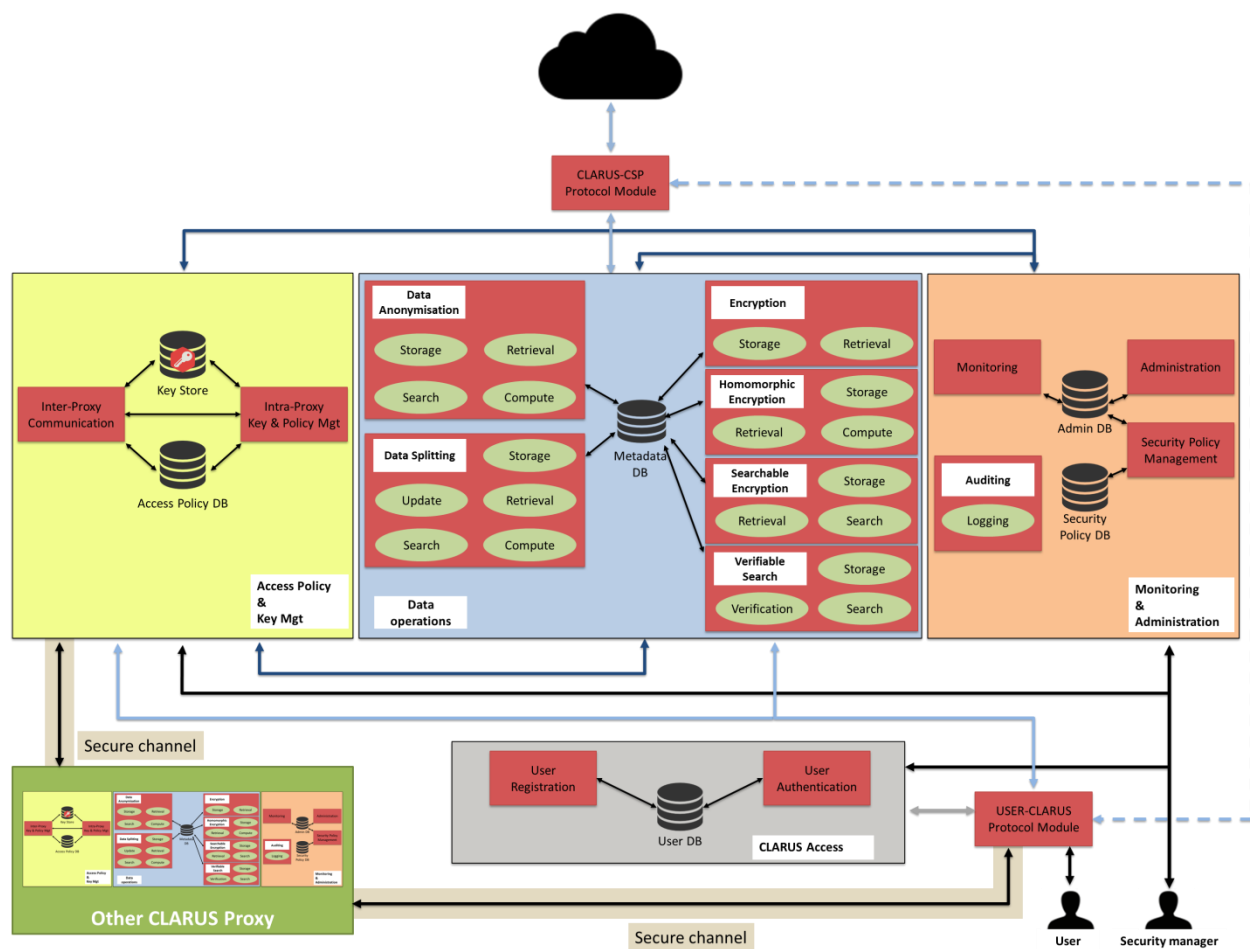
The work done in WP4 handled a complete specification of the architecture of the CLARUS platform and its internal modules. The first version of the architecture was described in Deliverable D4.1. It mainly specified the technical modules of the individual CLARUS proxy in charge of protecting the privacy of users' data for different data operations (storage, search, computation, retrieval). The resulting architecture consists of four main blocks:

- Access Policy & Key Management

- Data Operations
- Monitoring & Administration
- CLARUS Access

The design of the architecture for a single proxy directly bases on the requirements described in D2.2.

To support multiple CLARUS proxies a second version of the architecture was specified in D4.2. This document considered a more general scenario involving multiple CLARUS proxies that will interact with each other. This allows an organisation to authorise another organisation to perform some operations on their data and keep these under control. The second version of the architecture also includes the design of some new auditing services, which help the verification of the different operations. The design of the entire architecture is shown in the following figure:



This multiple CLARUS enabled architecture leads to some changes and additions. First of all, a secure channel to external CLARUS proxies was added. Since all data coming into a CLARUS proxy is filtered in the user CLARUS protocol module, an external CLARUS proxy must also access this module and send the requests through this module. This leads to some changes on the requirements for the user CLARUS protocol module, due to how the authentication and user identification for external CLARUS proxies has been handled. To keep things simple, the architecture was designed to distinguish between local “real” CLARUS users (users that are real persons) and technical users (such as database users). All local CLARUS users connect through the user CLARUS protocol module; then, the requests are handled only by an internal technical CLARUS user that is the same for all data operations. In case of an external CLARUS proxy accessing secured data, it is not planned to exhibit external user details. As a result, the external

CLARUS proxy needs a different authentication scheme, hence leading to some minor variations of the requirements for the authentication process.

The work in WP4 also described the development of a security as a service (SECaaS) infrastructure and concept for the CLARUS framework. This SECaaS differs from usual SECaaS business models like anti-virus software or intrusion detection services. While these security services are located in the trusted cloud, a main assumption of CLARUS is that there is no general trust against the cloud provider. The proposed SECaaS for CLARUS therefore consists of the possibility of securing and accessing data, but still being able to profit from cloud service providers, their hardware, services and offers and thus circumventing the contradictions.

In total five SECaaS services have been identified:

- Continuous monitoring and risk assessment of the platform
- CLARUS communication with the CSP
- CLARUS end user access to the CLARUS proxy
- Exchange in between multiple CLARUS proxy entities
- Provision of CLARUS secured data to an external user

While the first four services were described in detail in Deliverable D4.2, the fifth service covering the provision of CLARUS secured data to external users without a local CLARUS proxy instance was described in Deliverable D4.4. This service mainly bases on the creation of Secure Containers and the secure transmission of these containers to heterogeneous targeted devices owned by external entities. The secure containers thereby contain the ability for external users outside the trusted zone to access data that have been safeguarded by the obfuscation and encryption security primitives provided by CLARUS.

The introduction of the security service has been motivated by the health use case, to securely send patient records to an external specialist outside the trusted zone. The provided techniques allow this process and also take care of securing the provided data or the access information to this data respectively.

For this, the concept of Secure Containers was introduced, which makes use of existing and publically available basic technologies such as virtualization, migration and trusted execution environments. Secure Containers encapsulate a virtual machine and contain all necessary information to access the data or – according to the use case – the data themselves. A special focus was placed on the possibility of migrating this Secure Container to external devices. Because these devices might also have different hardware architectures, the provided solution can also cope with these differences. This aspect of the Security as a Service approach for CLARUS enhances the possibilities of CLARUS and allows for new use cases and options, while it also preserves the CLARUS core abilities.

2.1.3 Implementation (WP5)

This section summarizes the works achieved so far in the implementation phase of the project (WP5).

The WP5 implementation is actually divided in three tasks:

- the CLARUS interface
- the CLARUS modules
- the CLARUS platform deployment and integration

1.1.1.7 CLARUS interface

The definition of the CLARUS interface is the most advanced task of the implementation work package, being due M18. During the course of this task, choices were made regarding the implementation, notably the implementation of user interfaces.

It has thus been decided to develop :

- command-line interfaces for tasks performed occasionally by a single privileged user (e.g. the security manager or the IT team),
- web-based interfaces for tasks performed more frequently by a variety of users.

This could imply some modifications of the user interface requirements.

On the other hand, CSP services-preserving add-ons, aiming to preserve certain operations in the cloud, have been described in detail. We have specified the "compute with data splitting" add-on, the "homomorphic encryption" add-on and the "searchable encryption" add-on.

Another choice was needed regarding the protocol module, which is responsible for intercepting and parsing the requests and responses to the cloud service provider. Indeed, the protocol module should support both a *streaming* mode and a *buffering* mode, depending on whether the data could be stored *one at a time* or not. The potential impact on requirements is therefore linked to the WP3 results regarding data storage (as a whole dataset or record by record).

1.1.1.8 CLARUS modules

In parallel, WP5 partners have started implementing the different modules of the CLARUS platform specified in WP4 – notably the access policy, security management, data operations and protocol modules.

Regarding the protocol module, proof-of-concepts have been developed to evaluate the feasibility to intercept, parse and modify requests and responses for the PostgreSQL protocol.

Regarding the data protection modules, REST-based interfaces have been defined, as well as sequence diagrams detailing the different data protection mechanisms and functionalities. The modules for anonymization/coarsening and data splitting have already been implemented, as well as a first version of the searchable encryption module.

For some data operation modules, CLARUS provides an add-on that must be installed on the CSPs, i.e. a set of primitives that the CSP service must call as a result of a CLARUS request.

1.1.1.9 CLARUS platform deployment and integration

A cloud computing platform has been deployed in order to host the demonstrators. This platform allows the partners to create multiple environments to implement and test modules. Two special environments have been configured to host the use case demonstrators.

These demonstrators are targeting

- the publication of geo-referenced data on the Internet, i.e.
 - Geodata storage in a spatial database (PostgreSQL/PostGIS),
 - Web application for geodata publication (using WFS to request a Geoserver),
 - Web application for geo-processing (using WPS to request a 52°North server and Rserve)
 - Web application for collaboration on geodata (using WFS-T to request Geoserver)

- the eHealth use case, i.e. an application that allows to store and manage clinical data within the premises of a healthcare organization and, also, to offer (protected) data for clinical research (by the same organization or by third parties).

2.1.4 Evaluation, testing and validation (WP6)

The work package 6 is divided in three main tasks :

- Benchmark design, testing and validation
- Guidance on compliance with legal obligations
- Contribution to standards

1.1.1.10 Benchmark

The benchmark task consists in the design of a suite aligned with CLARUS' needs to gather metrics (such as CPU/Memory usage and bandwidth allocation), which will quantify the overhead (if any) of the CLARUS solution. These performance metrics are defined inside the Requirements Specification (REQ-NF_PERF-1.3).

A configuration management solution (related to the configuration and delivery management requirements) has been deployed. In addition, the tools and scenarios that will be used to gather and visualize performance metrics have been designed and chosen.

As soon as the modules are ready and deployed in the demonstration platform (WP5), the different protection methods (encryption, anonymization and data-splitting) will be benchmarked separately. All the benchmarks will be done using the demonstration cases in order to have realistic figures.

1.1.1.11 Compliance with Legal obligations

The task related to the guidance and compliance with legal obligations can be summed up as a guidance of the consortium through the most important aspects of the recently adopted General Data Protection Regulation (GDPR), which will become directly applicable in all EU Member States on the 25th of May 2018. Although the GDPR will not become law before the end of CLARUS project, it is important that partners take into account the already agreed upon provisions for future compliance. Also, KU Leuven referred extensively to the recent developments regarding data transfers to third countries, by analysing the upcoming 'Privacy Shield Agreement', which will replace the invalidated 'Safe Harbor Agreement'. Furthermore, in this addendum, KUL analyzed the issue of public access to cloud data, which was already addressed in deliverable 2.4, by providing examples of the recently adopted legislations in UK and France. Emphasis was given to Article 48 of the GDPR, which aims to prevent future search warrants by law enforcement authorities arbitrarily demanding disclosure of personal data stored in European data centers and requiring the recipient of the order to turn over information within its control, irrespective of the place in which the data is effectively stored.

1.1.1.12 Contribution with standards

The work done in this task was mainly about the coordination of the comments to the ETSI CSC phase 2 reports on cloud services interoperability. We also contributed to the DPSP and map of the standards in CLARUS.

2.2 Implications regarding the initial requirements (V1)

2.2.1 Implications regarding the use cases

1.1.1.13 End-user perspective

Storing dataset as a whole (vs. storing records one by one)

An important distinction arises from the description of the WP3 operations and the WP5 protocol module. This distinction concerns the way the CLARUS proxy deals with data storage depending on the selected protection technique.

We thus need to consider the two cases below:

- either the data can be stored record by record (*one at a time*),
- or the data need to be stored as a whole dataset (in a *monolithic* way). In this case, adding or removing records to a dataset already stored in the cloud only comes at a cost (depending on if this is supported by the protection technique).

This distinction needs to be clearly underlined in the present document, which leads us to a revamping of the end-user perspective (EUP) use cases:

- Store a dataset in the cloud (i.e. outsource a dataset)
- Add/update/remove records to an outsourced dataset (i.e. store data by id)
- Retrieve records from an outsourced dataset (i.e. retrieve data by id)
- Search for records in an outsourced dataset

1.1.1.14 Security management

Multi-proxy management and user registration

According to the results of the architecture design (WP4), the chosen option for managing several CLARUS proxies is *data routing* (as opposed to the *direct access* approach). This means that some of the use cases regarding multi-proxy management need to be modified, in particular the ones referring to the export/import of users to/from other proxies. Indeed, there is neither import nor export of *real* users with data routing.

Furthermore, an additional use case should be introduced to account for the data routing scenario. This use case will describe how a data owner (namely the “local CLARUS proxy”) allows a data consumer (namely the “external CLARUS proxy”) to access parts of its data.

Per-user security materials (CLARUS keys and metadata)

During the course of the project, a shift appeared in the terminology: the term “security materials”, initially used to generically designate any piece of information needed to apply a protection technique, is no longer used. We now refer to “CLARUS keys” and “CLARUS metadata”.

In addition, the research on security and privacy (WP3) led to say that there should not be an encryption key per user, but instead a single encryption key per proxy.

This means that the use cases dealing with per-user security materials need to be modified.

Per-user access level

Design of the data operation modules (WP5) led to the conclusion that it was not efficient to have a per-user access level (e.g. a percentage of accuracy on the retrieved results or a degree of coarsening/generalization).

Security policy

Regarding the definition of a security policy, the notion of “attribute type” (distinct from “data type”) has been introduced during the works on WP3.

An *attribute type* (which refers to the degree of sensitiveness of the attribute) can be non_confidential, confidential, identifier or quasi_identifier; while a *data type* (which strictly refers to the data the attribute encompasses) can be categoric, categoric_ordinal, numeric_discrete, numeric_continuous, geometric_object, etc.

Protection parameters and outcome accuracy

Following the works on WP3 about security policy definition, the concept of outcome accuracy has been integrated into the more generic concept of protection parameter (e.g. microaggregation level, degree of data coarsening for the dataset, etc.).

Dataspaces and datasets (terminology)

During the course of the project, it was also important to emphasize the difference between a *dataspace* and a *dataset*. A special emphasis is placed in this document on the difference between these two terms, notably in the description of the security management use cases.

2.2.2 Implications regarding functional requirements*1.1.1.15 Implications regarding business rules***Data utility preservation**

Given the intermediate results from the research on new security techniques (WP3), it results that:

- anonymization implies a loss of utility (data accuracy) for CSPs and end-users;
- data splitting implies (partial) loss of utility for CSPs, but preserves full utility for end-users on all operations;
- encryption implies (full) loss of utility for the CSPs, but preserves full utility for end-users on the supported operations;

Efficiency preservation

Given the intermediate results from the WP3, it also appears that a loss of efficiency is inevitable up to a degree for some data protection mechanisms and data operations. In order to take this into account, the requirements about loss of efficiency needed to be reformulated.

*1.1.1.16 Implications regarding features***Access control***User import from another proxy*

As a result of the multi-proxy scenario we envision (WP4), importing/exporting *real* users from an external proxy is not needed anymore. Thus the section “import users from another proxy” needs to be removed, or renamed into a more generic section “register an external proxy”.

User activation flag

The notion of “activation flag (YES/NO)” introduced in the requirements V1 was closely linked to the mechanisms of inter-proxy user management. For instance, upon deactivation of a shared user at the local CLARUS proxy, a sequence of actions was specified aiming at notifying the external CLARUS proxy, and at triggering on its side the process of user deactivation.

With the *data routing* option for managing several CLARUS proxies, which was chosen in the course of the architecture design (WP4), this workflow is not needed anymore. Consequently, it is not necessary to make mention of an activation flag. The corresponding requirements need to be updated accordingly.

Register/unregister users

Even though the process of activating/deactivating users is no longer needed with the inter-proxy user management we plan, it is however still required to associate CLARUS users with respect to the configured user repository (either the CLARUS-embedded repository or the company LDAP). To do so, we use the term “registration”.

This registration process is actually required for enrolling not only *real* users, but also *technical* users (identifying e.g. external proxies, database users, generic accounts, etc.).

It is important to note that this registration status is kept only for the sake of access control; it is not intended to trigger any process. The automatic deletion of data following unregistration is indeed out of scope.

1.1.1.17 Implications regarding user interface

Console interface vs. Web interface

Given the intermediate results of the implementation work package (WP5), the user interface requirements need to be modified in order not to anticipate implementation choices. In particular, it has been stated in the course of WP5 that any user interface accessible only by the security manager (who is the most advanced user of CLARUS and who is fully knowledgeable on its configuration) should be a console interface instead of a web interface. Any reference to screens, dashboards, menus, buttons, etc. should therefore be removed in this document and replaced by more generic requirements allowing the implementation partners to choose the preferred solution.

Categories of user interfaces

In addition, the user interfaces need to be divided in :

- Security policy management
- Access policy management
- Administration
- Monitoring and attack management
- Protected data viewer

2.2.3 Implications regarding non-functional requirements

At this point in time, and since the evaluation and validation phase (WP6) is still at an early stage, there is no implication identified so far regarding non-functional requirements.

3 Refined use cases

3.1 Reminders

3.1.1 CLARUS actors

The actors are described here in the context of the CLARUS solution. It is possible that some of these actors may be involved in other activities within their organizations (for instance the IT team, the Security Manager, etc.), but these activities are not within the scope of this Section.

The different actors involved in the CLARUS solution can be divided into two main groups:

- Stakeholders in the infrastructure management,
- End-users.

1.1.1.18 Actors involved in Infrastructure Management

The **IT team / Administrator** is responsible for installing CLARUS in the Information System and configuring it with respect to the infrastructure.

The **Architect** is responsible for the design of the solution to be implemented by the development team.

The **Developer**, also referred to as “IT Development team”, is responsible for the development of additional modules, extending CLARUS functionalities in order to support protocols, data types, functions, etc. that are not natively supported by the solution.

The **Cloud Service Provider** is the company/organisation providing an IaaS, PaaS and/or SaaS type of service that is used by the Data Providers and Data Consumers, when storing and accessing data in/from the cloud through the CLARUS proxy. In the CLARUS context, the cloud service providers are considered “*Honest but curious*”, meaning that they correctly execute the operations but they might exploit at the information they store.

1.1.1.19 End Users

The **Security Manager** is in charge of managing access control of CLARUS users and defining security policies regarding data usages, data access protocols, data types and expected outcome accuracy. This is the user that probably interacts the most (and in the most advanced way) with CLARUS through a user interface.

The **Data Provider**, also referred to as “Data owner”, is the one that when the notion of data ownership is at stake, wants to securely store data in the cloud through the CLARUS proxy, with the guarantee that the data will not be disclosed to the cloud service provider. Data providers may also want to update their data or the data of other data providers (given that they have the proper authorizations) through CLARUS.

The **Data Consumer** is the potential user of the data stored in the cloud services through the CLARUS solution. Data consumers may need to access and retrieve these data, or to perform an attribute-based search or aggregated calculations, depending on whether they want to consult a specific record or to do research studies. Data consumers may also want that their queries or the result of their queries will not be disclosed to the cloud service provider or any third-party (as long as the protection technique allows it).

3.1.2 CLARUS main scenarios

In this section we focus on the CLARUS use cases. We define the **main scenarios** of use of the CLARUS solution. We use UML use case diagrams to define CLARUS software functional requirements in relation to the scenarios.

Depending on the considered scenario, CLARUS can be regarded as:

- A proxy running on fully trusted local premises which is transparent for the local applications and the end-users,.
- A PaaS/SaaS service deployed on CSPs adopting CLARUS to properly manage/operate encrypted or split data via well-defined APIs. This implementation integrating CLARUS libraries in the cloud will be subsequently referred to as “CLARUS inside”.

The following scenarios are derived mainly from our use cases [A5], but also from more generic use cases in fields other than than geo-publication or e-Health. This allows us the identification of the following **features**, which CLARUS needs to support and implement by using privacy-preserving techniques:

- Store a whole dataset,
- Store data by id,
- Retrieve data by id,
- Search for data by attributes,
- Update/remove data,
- Compute in the cloud.

Combining these features together we propose the following **main scenarios** that reflect the different use cases of the CLARUS solution, from the data provider/consumer perspective :

- Scenario 1 : Store data in the cloud,
- Scenario 2 : Search in the cloud,
- Scenario 3 : Update in the cloud,
- Scenario 4 : Computation in the cloud.

Each of these scenarios is illustrated with an UML use case diagram, representing the relationships between the main actors and the “system” embodied by the CLARUS solution to implement. The use cases describe how the system interacts with the different elements around it: users/actors, other systems, etc.

In addition scenarios showing perspectives other than the data provider/consumer are described, notably:

- CLARUS installation,
- Access control management,
- Security policy management,
- Monitoring.

Finally, the different scenarios refer to the notion of “dataspace”. A dataspace is a virtual space that links the applications/services running on IT premises with the cloud storage services and that protects the data according to a security policy. A dataspace provides an internal endpoint that supports the protocol used by the applications/services running on IT premises to access the outsourced datasets. All outsourced datasets within a dataspace are protected with one security policy. Protecting two datasets with two different security policies requires defining two dataspace.

3.1.3 Conventions related to the CLARUS use cases

1.1.1.20 Definitions

Primary actor requests the system to deliver a goal (usually represented on the left of the use case diagram).

Secondary actor is an actor from which the system needs assistance to achieve the primary actor's goal (usually represented on the right of the use case diagram).

Main success scenario is the typical and illustrative sequence of actions and interactions leading to a goal delivery by the system, even though it may not be the only success path.

Alternate scenario (Variation) is any other scenario or scenario fragment written as an extension to the main success scenario.

1.1.1.21 Naming Scheme

Each use case involves an identifier <Use Case Id> defined by the following nomenclature :

<Use Case Id> = UC-<Classification code>-<Progressive number>
--

Figure 1 - Use Case ID Naming Scheme

The classification codes refer to some criteria according to the following classification.

1.1.1.22 Classification codes

The use cases are classified according to:

- End-User Perspective (EUP),
- Security Management (SEC),
- Monitoring (MON),
- Administration (ADM),
- Design and Implementation (IMPL)

3.2 End-user perspective

3.2.1 Outsource a dataset to the cloud (store and retrieve)

1.1.1.23 Overview

This scenario considers a data provider (authenticated by a CLARUS proxy) who wants to outsource his/her dataset to the cloud and who wants to share it with a data consumer (authenticated by a CLARUS proxy). When we refer to the expression “outsourcing a dataset”, we actually refer to three operations :

- Storing a dataset in the cloud (i.e. *upload* a dataset)
- Retrieving a dataset from the cloud (i.e. *download* a dataset)
- Deleting a dataset stored in the cloud

These operations are performed on the dataset as a whole (i.e. in a *monolithic* way), as opposed to operations performed on single records, one at a time (cf. section “Update an outsourced dataset” 3.2.3).

System

In this scenario the system is the CLARUS proxy running on trusted local premises. One or several CSP(s) may be involved, but only for the purpose of data storage. Consequently, there is no CLARUS software needed on the cloud.

UML diagrams

The UML diagram below represents the corresponding use cases (and actors).

In this scenario a data provider uploads, downloads and deletes his/her datasets on the cloud. A data consumer downloads a shared dataset.

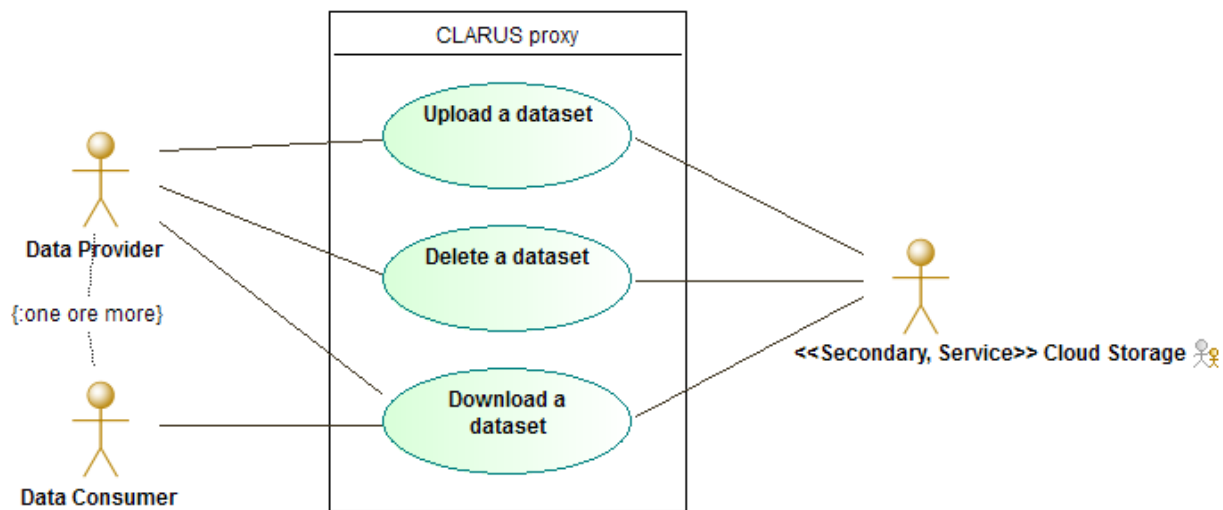


Figure 2 - Outsource Dataset Use Case Diagram

The UML diagram below represents upload, download and delete protected dataset:

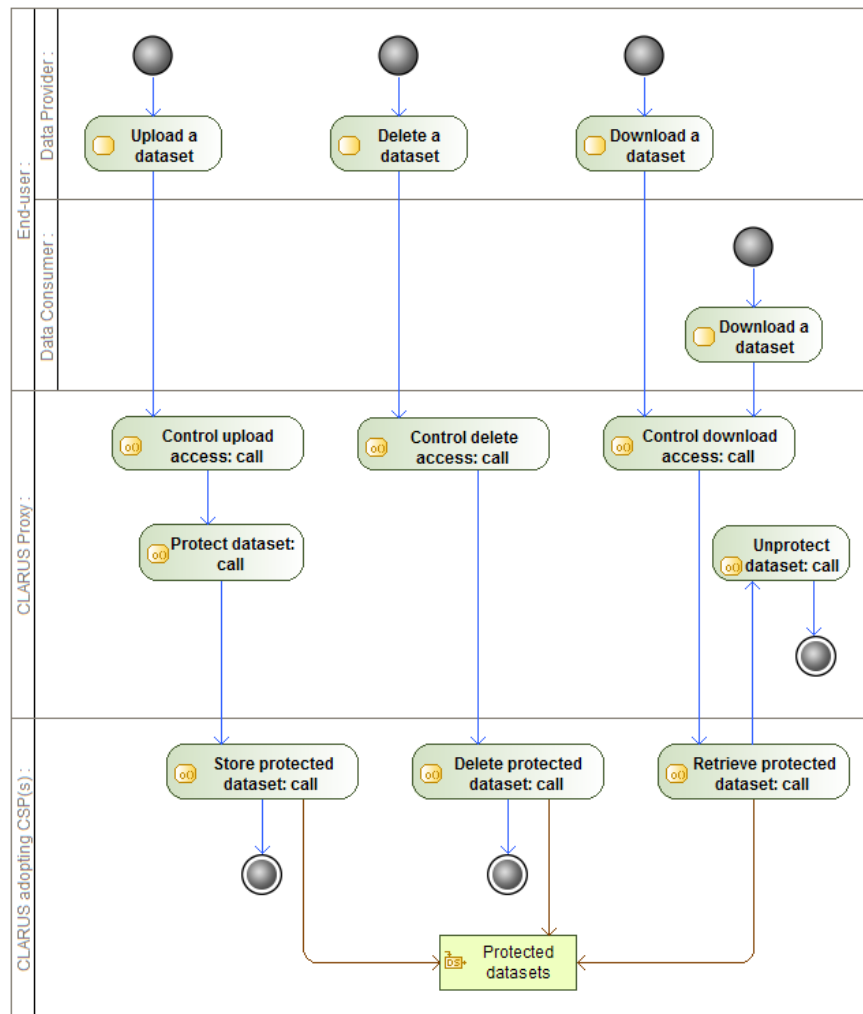


Figure 3 - Outsource Dataset Activity Diagram

1.1.1.24 Use cases

Upload a dataset (UC-EUP-01)

UC-EUP-01	<i>Upload a dataset (store)</i>
Actors	Data provider, cloud storage service (secondary)
Description	The data provider wants to securely store a dataset in the cloud through the CLARUS proxy and using the storage service(s) provided by the CSP(s), having the guarantee that these data will not be disclosed to the CSP(s)
Pre-conditions	
[Pre-condition 1]	The dataset to store in the cloud is either structured (e.g. a table in a database) or unstructured (a set of files)
[Pre-condition 2]	The IT team has set up a user account in one (or many) cloud service provider(s)
[Pre-condition 3]	The security manager has set up a security policy for the dataspace
[Pre-condition 4]	The data provider is authorized to upload a dataset (write) within the dataspace
Main scenario	
Step 1	The data provider , via a dedicated application, requests an upload of his/her dataset in the cloud through the CLARUS proxy

Step 2	The system identifies the dataspace in which the dataset will be stored
Step 3	The system controls that the data provider is authorized to upload a dataset (according to the access rights defined in his/her profile) within the dataspace
Step 4	The system protects the dataset as a whole (in a <i>monolithic</i> way) according to the security policy defined for the dataspace
Step 5	The system stores the protected dataset on the cloud using the cloud storage service(s) provided by the CSP(s)
Step 6	The cloud storage service(s) confirms the system about the success of the storage operation
Step 7	The system confirms the data provider about the success of the storage operation
Post-conditions	
[Post-condition 1]	The protected dataset is stored in the cloud

Download a dataset (UC-EUP-02)

UC-EUP-02	<i>Download a dataset (retrieve)</i>
Actors	Data provider, data consumer, cloud storage service (secondary)
Description	The data provider (or the data consumer) wants to retrieve a whole dataset stored in the cloud, through the CLARUS proxy, having the guarantee that the data are not disclosed to the CSP(s)
Pre-conditions	
[Pre-condition 1]	The dataset has previously been protected and uploaded to the cloud through the proxy
[Pre-condition 2]	The data provider (or the data consumer) is authorized to download the dataset (read)
Main scenario	
Step 1	The data provider (or the data consumer), via a dedicated application, requests a download of the dataset from the cloud
Step 2	The system identifies the dataspace from which the dataset will be retrieved
Step 3	The system controls that the data provider (or the data consumer) is authorized to download a dataset (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset) from the dataspace
Step 4	The system retrieves the protected dataset from the cloud using the cloud storage service(s) provided by the CSP(s)
Step 5	The system unprotects the dataset as a whole (in a <i>monolithic</i> way) according to protection set initially
Step 6	The system provides the data provider (or the data consumer) with a clear (unprotected) dataset
Post-conditions	
[Post-condition 1]	The data provider (or the data consumer) has retrieved a clear (unprotected) dataset without disclosure to the CSP(s)

Delete a dataset (UC-EUP-03)

UC-EUP-03	<i>Delete a dataset</i>
Actors	Data provider, cloud storage service (secondary)

Description	The data provider wants to delete a dataset already stored in the cloud through the CLARUS proxy, having the guarantee that the data are properly deleted by the CSP(s)
Pre-conditions	
[Pre-condition 1]	The dataset has previously been protected and stored in the CSP(s) through the proxy
[Pre-condition 2]	The data provider is authorized to delete the existing dataset (write)
Main scenario	
Step 1	The data provider , via a dedicated application, requests a deletion of a dataset outsourced in the cloud
Step 2	The system identifies the dataspace corresponding to the dataset to be deleted
Step 3	The system controls that the data provider is authorized to delete the dataset (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The system deletes the protected dataset from the cloud using the cloud storage service(s) provided by the CSP(s)
Step 5	The cloud storage service(s) confirms the system about proper deletion of the protected dataset
Step 5	The system confirms the data provider about proper deletion of the protected dataset
Post-conditions	
[Post-condition 1]	The data provider has deleted the outsourced dataset: these data are not in the cloud any more

3.2.2 Search for records in an outsourced dataset

1.1.1.25 Overview

This scenario considers a data provider (authenticated by a CLARUS proxy) who has published his/her dataset on the cloud. Any data provider (and any data consumer) can search data by means of an identifier or by means of attributes through the CLARUS proxy, and any user can search data directly accessing the cloud search service.

In the CLARUS project, direct search in the cloud (without going through the CLARUS proxy) is only considered for anonymized datasets (and non-protected datasets).

With data anonymization, *real* data identifiers need to be removed (because they may univocally re-infer the individual) and replaced with random identifiers for maintaining the coherence of the database. Thus, “direct search” is only supported for search by means of attributes and not for search by means of an identifier.

System

In the search scenario the system encompasses:

- The CLARUS proxy running on trusted local premises,
- The CLARUS adopting CSP(s).

Only one CLARUS proxy is involved. The CSP(s) is(are) involved to search data by an identifier or by attributes: consequently, CLARUS software might be needed on the cloud side, namely “CLARUS inside”.

UML diagrams

The UML diagrams below represent the corresponding use cases (and their actors) with two scenarios.

In a first *standard* scenario data access is done by searching for records **through the CLARUS proxy** on a protected dataset.

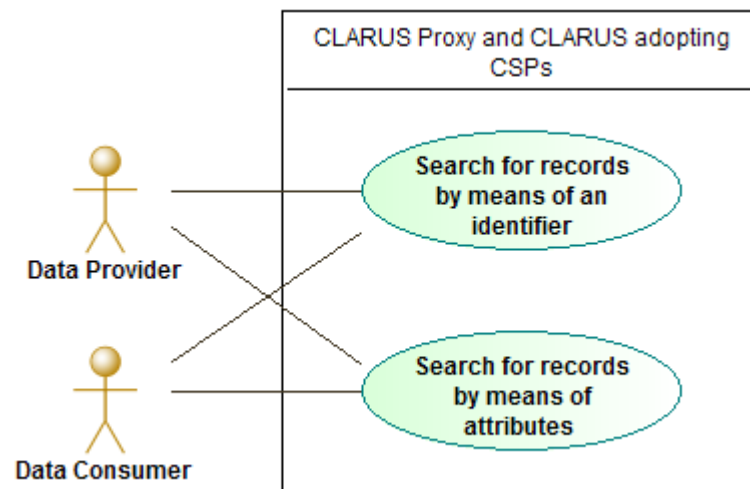


Figure 4 - Search for Records Through Proxy Use Case Diagram

In a second scenario data search is done directly in the cloud, without need of the CLARUS proxy.

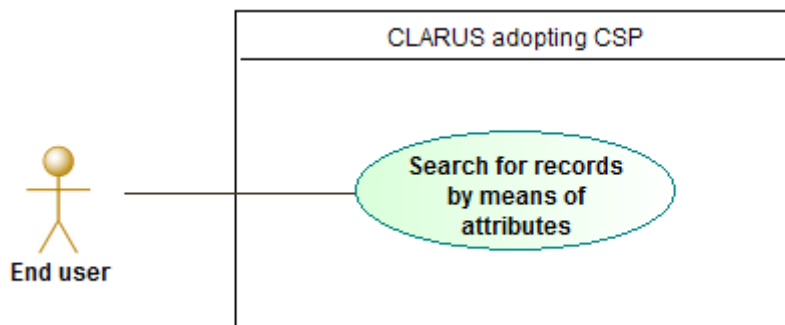


Figure 5 - Search for Records Directly in the Cloud Use Case Diagram

The UML diagram below represents search on a protected dataset either through the CLARUS proxy or directly in the cloud:

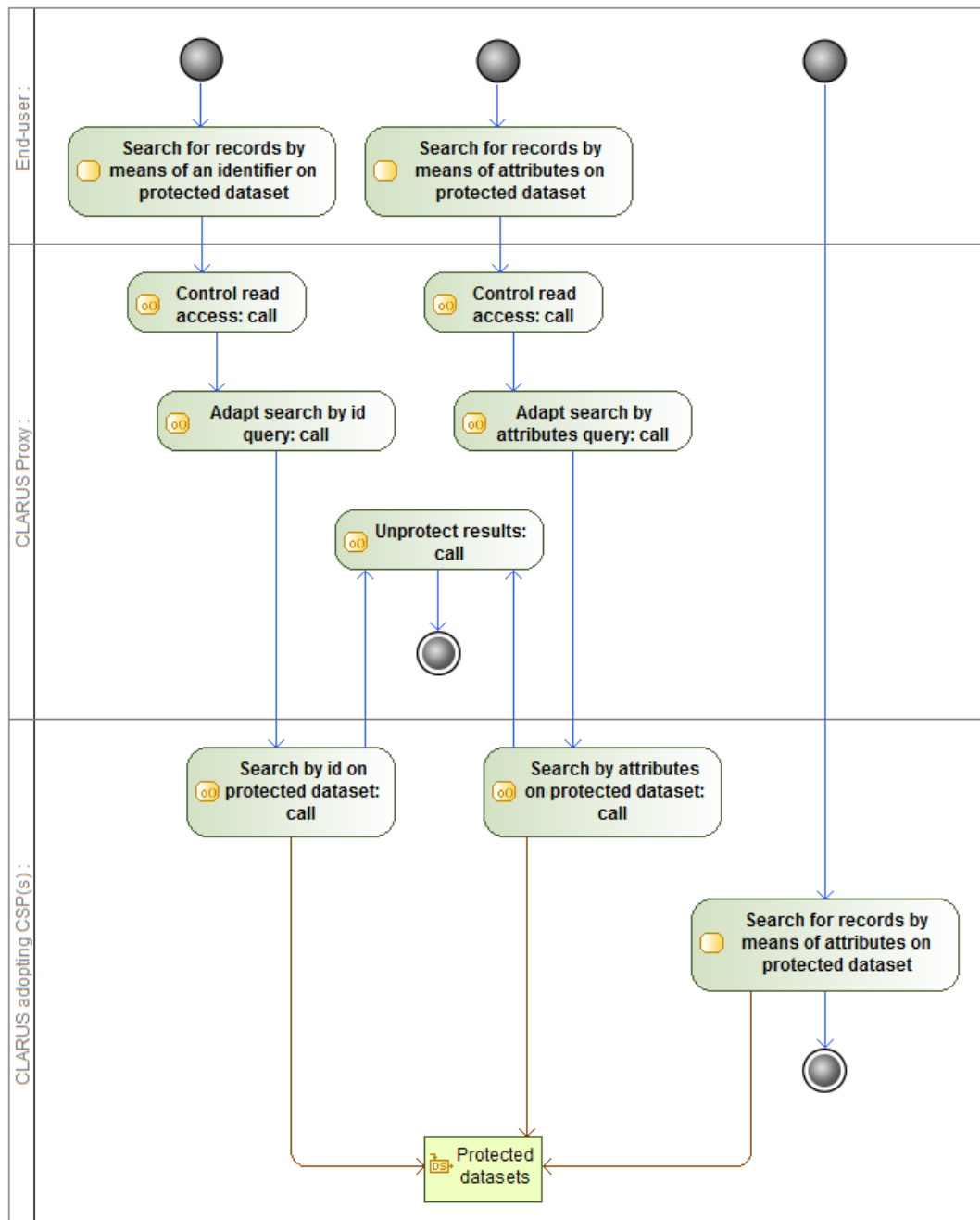


Figure 6 - Search for Records Activity Diagram

1.1.1.26 Use cases

Search for records by means of an identifier (UC-EUP-04)

UC-EUP-04	<i>Search for records by means of an identifier (retrieve by id)</i>
Actors	Data provider, data consumer, end-user, cloud storage service (secondary)
Description	The data provider (or the data consumer) wants to retrieve data from the cloud through the CLARUS proxy, having the guarantee that the data are not disclosed to the CSP(s) or any other unauthorized party
Pre-conditions	
<i>[Pre-condition 1]</i>	The dataset has previously been protected and stored in the CSP(s) through the proxy

<i>[Pre-condition 2]</i>	The data provider (or the data consumer) is authorized to search data in the existing dataset (read)
Main scenario	<i>Data provider (or consumer) searches for data by id, through the proxy</i>
Step 1	The data provider (or the data consumer), via a dedicated application, requests retrieval of data from the outsourced dataset, by means of their id
Step 2	The system identifies the dataspace and the outsourced dataset from which the data will be retrieved
Step 3	The system controls the data provider (or the data consumer) is authorized to retrieve existing data (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The proxy adapts the query according to the protection set initially
Step 5	The system retrieves the protected data from the cloud using the cloud storage service(s) provided by the CSP(s), by means of their id
Step 6	The system unprotects the data according to protection set initially
Step 7	The system provides the data provider (or the data consumer) with clear (unprotected) data
Post-conditions	
<i>[Post-condition 1]</i>	The data provider (or the data consumer) has retrieved clear (unprotected) data without disclosure to the CSP(s)

Search for records by means of attributes (UC-EUP-05)

UC-EUP-05	<i>Search for records by means of attributes</i>
Actors	Data provider, data consumer, end-user, cloud storage service (secondary)
Description	The data provider wants to search for data by attributes, having the guarantee that the content of his/her query will not be disclosed to the CSP(s) or any other unauthorized party
Pre-conditions	
<i>[Pre-condition 1]</i>	The dataset has previously been protected and stored in the CSP(s) through the proxy
<i>[Pre-condition 2]</i>	The data provider (or the data consumer) is authorized to search data in the existing dataset (read)
Main scenario	<i>Data provider (or consumer) searches for data by attributes, through the proxy</i>
Step 1	The data provider (or the data consumer), via a dedicated application, requests a search for data in the cloud, by means of an attribute-based query
Step 2	The system identifies the dataspace and the outsourced dataset from which the data will be searched
Step 3	The system controls that the data provider (or the data consumer) is authorized to search for data (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The proxy adapts the query according to the protection set initially
Step 5	The proxy invokes the search service(s) provided by the CSP(s)
Step 6	The search service(s) provided by the CSP(s) search and return the protected data to the proxy
Step 7	The proxy unprotects the data according to the protection set initially
Step 8	The proxy provides the data provider (or the data consumer) with the clear

	(unprotected) data he/she requested
Variation 1	<i>End-user searches for data by attributes that have not been protected, directly in the cloud</i>
Step 1	The end-user accesses one of the CSP(s) to search for data in the cloud, by means of a clear (unprotected) attribute-based query
Step 2	The search service provided by the CSP provides the end-user with a correct search result. The correct search result may contain protected data
Variation 2	<i>End-user searches for data by attributes that have been protected, directly in the cloud</i>
Step 2	The search service provided by the CSP provides the end-user with a search result that may be incorrect. The incorrect search result either is empty or contains invalid (but protected) data
Post-conditions	
[Post-condition 1]	In case of search through the proxy, the data provider (or the data consumer), is provided with the clear (unprotected) search result
[Post-condition 2]	The clear (unprotected) search result are not disclosed to the CSP(s) or to any other unauthorized party
[Post-condition 3]	In case of search through the proxy, the clear (unprotected) query is not disclosed to the CSP(s) or to any other unauthorized party
[Post-condition 4]	In case of direct search in the cloud by means of attributes that have not been protected, the end-user is provided with a correct record set (even though some record attributes are protected)
[Post-condition 5]	In case of direct search in the cloud by means of attributes that have been protected, the end-user is provided with a record set that <i>may</i> be different from the one that would be retrieved from the original data (and, also, some attributes may be protected).

3.2.3 Update an outsourced dataset

1.1.1.27 Overview

This scenario considers one or more data provider(s) (authenticated by a CLARUS proxy) who wants to add and/or modify and/or remove records to an outsourced dataset.

Two alternatives may be considered:

- A single data provider (or multiple users seen as a single data provider by the CLARUS proxy) who wants to store his/her data on the cloud and modifies them at any time through CLARUS.
- Multiple data providers (identified by the CLARUS proxy) who store and share their data on the cloud and intend to modify them at any time.

System

In this scenario the system is the CLARUS proxy running on trusted local premises.

One or more CLARUS proxy(ies) is(are) involved. One or several CSP(s) may be involved, but only for the purpose of data storage. Consequently, there is no CLARUS software needed on the cloud.

UML diagrams

The UML diagram below represents the corresponding use cases (and their actors).

In this scenario a data provider adds, removes, and modifies records in his/her dataset on the cloud.

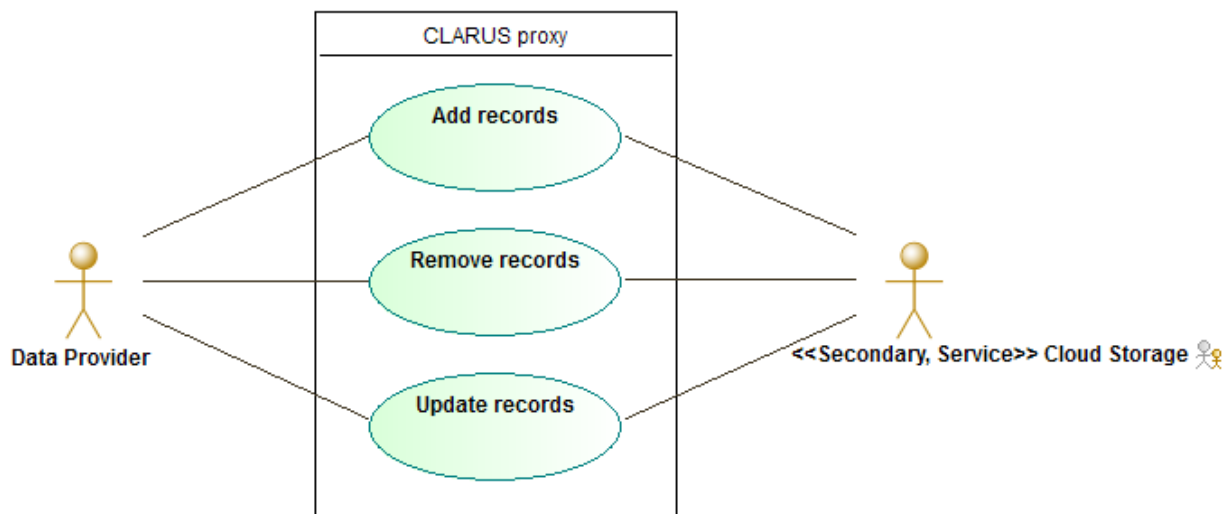


Figure 7 – Update an Outsourced Dataset Use Case Diagram

The UML diagram below represents add, remove and modify records on protected dataset:

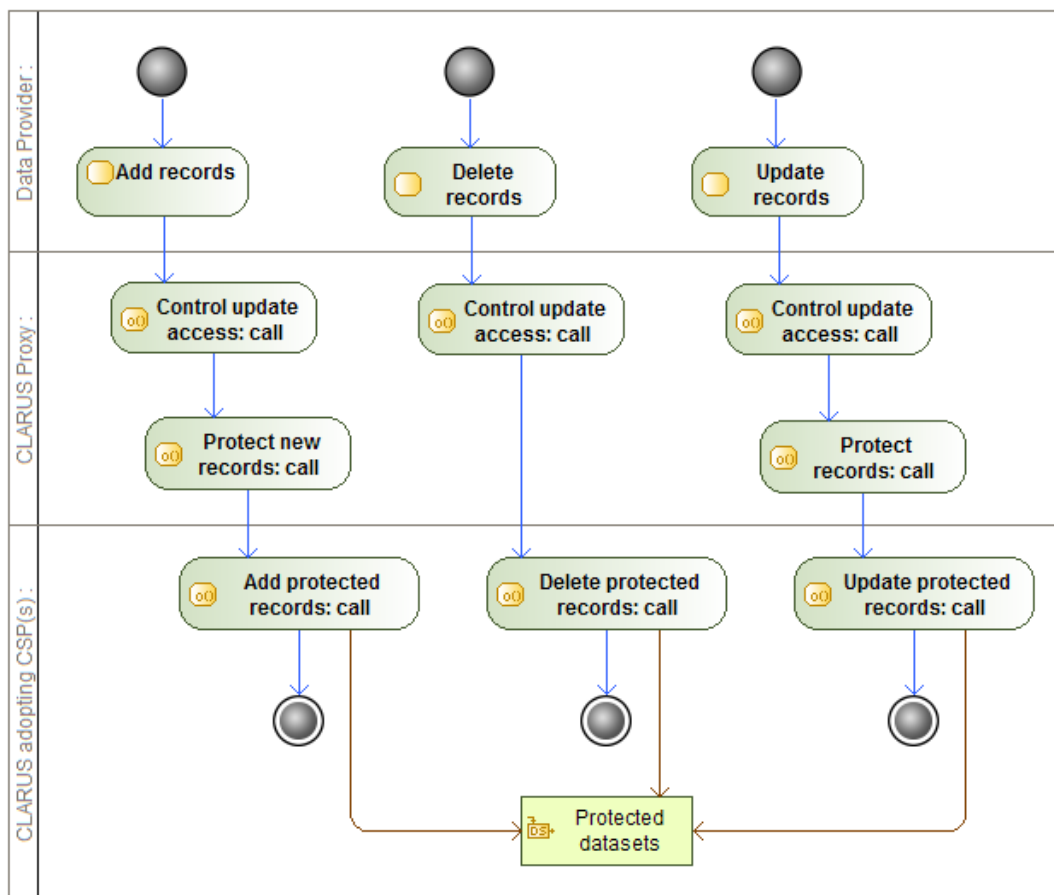


Figure 8 - Update an Outsourced Dataset Activity Diagram

1.1.1.28 Use cases

Add records to an outsourced dataset (UC-EUP-06)

UC-EUP-06	<i>Add records to an outsourced dataset (store by id)</i>
Actors	Data provider, cloud storage service (secondary)
Description	The data provider wants to securely add new records to an existing dataset in the cloud through the CLARUS proxy and using the storage service(s) provided by the CSP(s), having the guarantee that these data will not be disclosed to the CSP(s)
Pre-conditions	
[Pre-condition 1]	The dataset to modify have previously been protected and stored in the CSP(s) through the proxy
[Pre-condition 2]	The data provider is authorized to create new data in the existing dataset (write)
Main scenario	<i>Add records</i>
Step 1	The data provider , via a dedicated application, requests creation of new data in the outsourced dataset
Step 2	The system identifies the dataspace and the outsourced dataset in which the data to create will be stored
Step 3	The system controls that the data provider is authorized to create new data (according to the access rights defined in his/her profile) within the dataspace
Step 4	The system controls that the security policy defined for the dataspace allows modifications of the outsourced dataset
Step 5	The system protects the data according to the security policy defined for the dataspace
Step 6	The system stores the protected data on the cloud using the cloud storage service(s) provided by the CSP(s)
Step 7	The cloud storage service(s) confirm the system about the success of the storage operation
Step 8	The system confirms the data provider about the success of the storage operation
Variation 1	<i>Modifications of the outsourced dataset is incompatible with the security policy defined for the dataspace</i>
Step 5	The system informs the data provider about the failure of the storage operation
Post-conditions	
[Post-condition 1]	If modification is compatible with the security policy defined for the dataspace, new protected data are added in the outsourced dataset on the cloud
[Post-condition 2]	If modification is incompatible with the security policy defined for the dataspace, the outsourced dataset is not modified

Remove records from an outsourced dataset (UC-EUP-07)

UC-EUP-07	<i>Remove records from an outsourced dataset (delete by id)</i>
Actors	Data provider, cloud storage service (secondary)
Description	The data provider wants to securely delete records in an existing dataset in the cloud through the CLARUS proxy, having the guarantee that the data are properly deleted by the CSP(s)
Pre-conditions	

[Pre-condition 1]	The dataset to modify have previously been protected and stored in the CSP(s) through the proxy
[Pre-condition 2]	The data provider is authorized to delete data in the existing dataset (write)
Main scenario	<i>Remove records</i>
Step 1	The data provider , via a dedicated application, requests a deletion of records in a dataset outsourced in the cloud, by means of their id
Step 2	The system identifies the dataspace and the outsourced dataset from which the data will be deleted
Step 3	The system controls that the data provider is authorized to delete existing data (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The system controls that the security policy defined for the dataspace allows modifications of the outsourced dataset
Step 5	The system deletes the protected data from the cloud using the cloud storage service(s) provided by the CSP(s), by means of their id
Step 6	The cloud storage service(s) confirm the system about proper deletion of the protected data
Step 7	The system confirms the data provider about proper deletion of the protected data
Variation 1	<i>Modifications of the outsourced dataset is incompatible with the security policy defined for the dataspace</i>
Step 5	The system informs the data provider about the failure of the storage operation
Post-conditions	
[Post-condition 1]	If modification is compatible with the security policy defined for the dataspace, the data provider has deleted records in an outsourced dataset, by means of their identifier: these data are not in the cloud any more
[Post-condition 2]	If modification is incompatible with the security policy defined for the dataspace, the outsourced dataset is not modified

Update records in an outsourced dataset (UC-EUP-08)

UC-EUP-08	<i>Update records in an outsourced dataset</i>
Actors	Data provider, cloud storage service (secondary)
Description	The data provider wants to securely update existing records in an existing dataset in the cloud through the CLARUS proxy and using the storage service(s) provided by the CSP(s), having the guarantee that the data will not be disclosed to the CSP(s)
Pre-conditions	
[Pre-condition 1]	The dataset to modify have previously been protected and stored in the CSP(s) through the proxy
[Pre-condition 2]	The data provider is authorized to update data in the existing dataset (write)
Main scenario	<i>Update records</i>
Step 1	The data provider , via a dedicated application, requests an update of existing records in a dataset outsourced in the cloud, through the CLARUS proxy (by means of their identifier)
Step 2	The system identifies the dataspace and the outsourced dataset in which the data will be updated

Step 3	The system controls the data provider is authorized to update existing data (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The system controls that the security policy defined for the dataspace allows modifications of the outsourced dataset
Step 5	The system protects the data according to the security policy defined for the dataspace
Step 6	The system stores the protected data on the cloud using the cloud storage service(s) provided by the CSP(s)
Step 7	The cloud storage service(s) confirm the system about the success of the storage operation
Step 8	The system confirms the data provider about the success of the storage operation
Variation 1	<i>Modifications of the outsourced dataset is incompatible with the security policy defined for the dataspace</i>
Step 5	The system informs the data provider about the failure of the storage operation
Post-conditions	
[Post-condition 1]	If modification is compatible with the security policy defined for the dataspace, data are protected and updated in the outsourced dataset on the cloud
[Post-condition 2]	If modification is incompatible with the security policy defined for the dataspace, the outsourced dataset is not modified

3.2.4 Compute over an outsourced dataset

1.1.1.29 Overview

This scenario considers a data provider (authenticated by a CLARUS proxy) who has published his/her dataset on the cloud. Any data provider (and any data consumer) can request for a computation through the CLARUS proxy, and any user can request for a computation directly accessing the cloud computation service.

System

In this scenario the system encompasses:

- The CLARUS proxy running on trusted local premises,
- The CLARUS adopting CSP(s).

Only one CLARUS proxy is involved. The CSP(s) is(are) involved to store the data and to execute the computation process: consequently, CLARUS software might be needed on the cloud side, namely "CLARUS inside".

UML diagrams

The UML diagrams below represent corresponding use cases (with their actors) with two different scenarios. In both cases computation services are hosted in the cloud.

In the first scenario, computation services are performed on protected data through the CLARUS proxy. The respective results being protected need to be unprotected on the CLARUS proxy side.

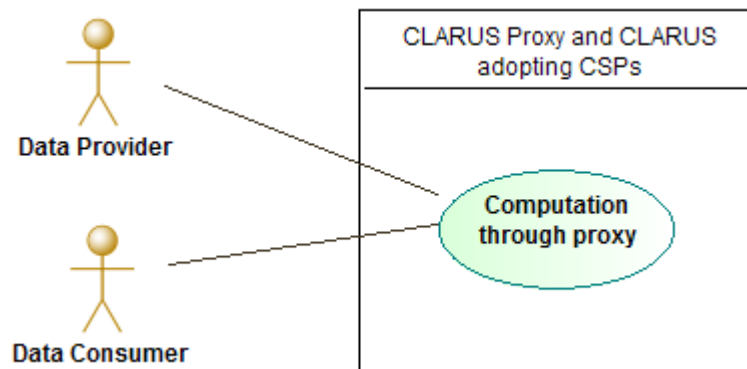


Figure 9 - Computation Through Proxy Use Case Diagram

In a second scenario any user can request for a computation directly accessing the cloud process service, without need of the CLARUS proxy.

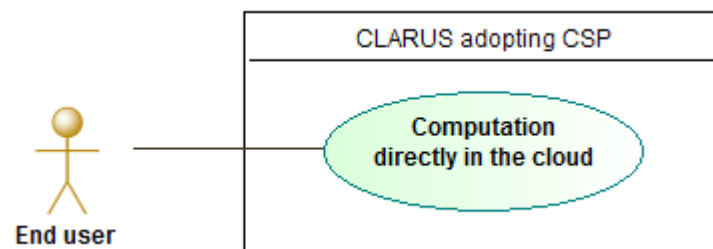


Figure 10 - Computation Directly in the Cloud Use Case Diagram

The UML diagram below represents computation on protected data either through the CLARUS proxy or directly in the cloud:

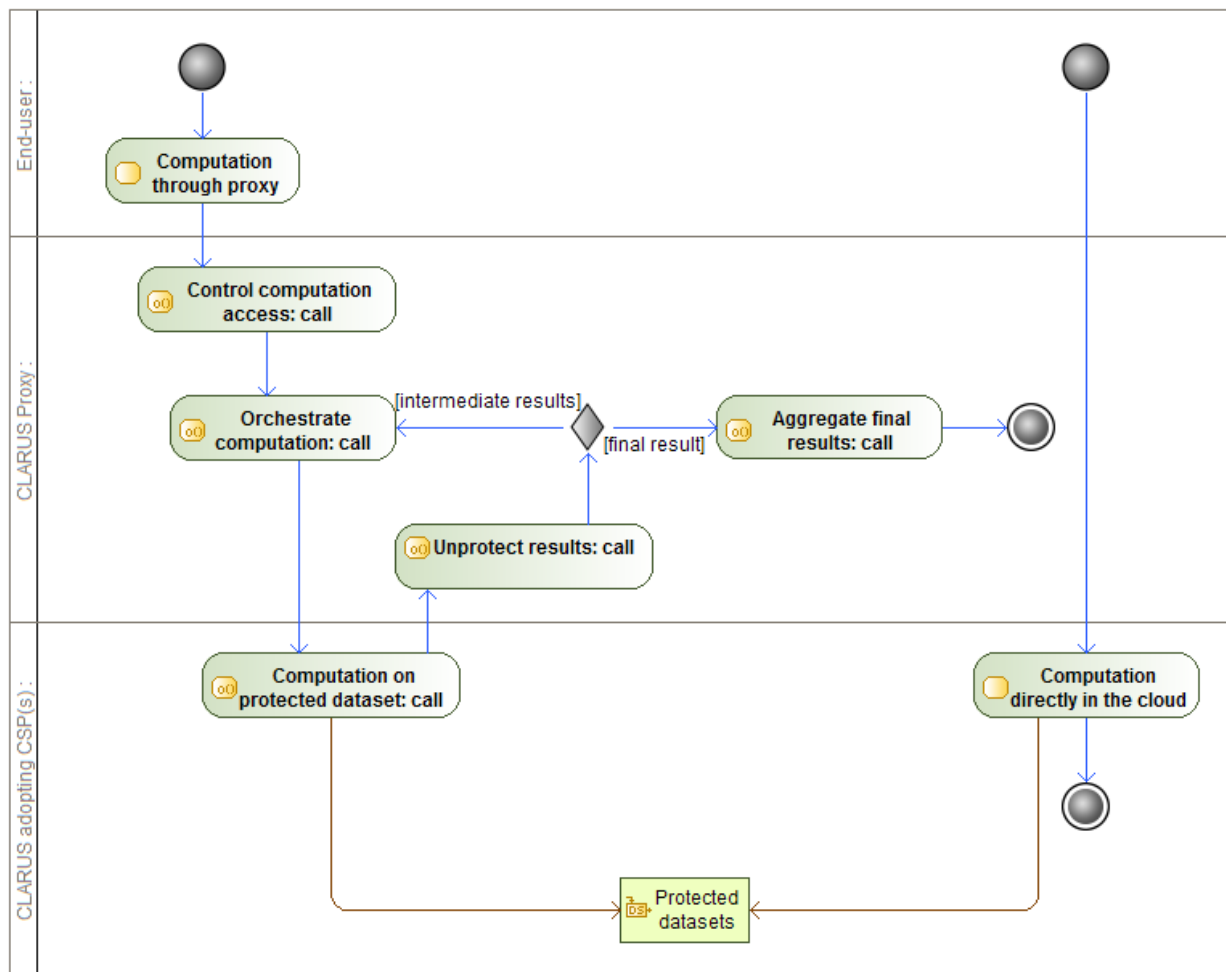


Figure 11 - Computation Activity Diagram

1.1.1.30 Use cases

Computation through proxy (UC-EUP-09)

UC-EUP-09	<i>Computation through proxy</i>
Actors	Data consumer, data provider
Description	The data provider (or data consumer) wants to perform a computation over protected data in the cloud, having the guarantee that his/her query and the response will not be disclosed to the CSP(s) or any other unauthorized party
Pre-conditions	
[Pre-condition 1]	The dataset have previously been protected and stored in the CLARUS-enabled CSP(s)
[Pre-condition 2]	The data provider (or the data consumer) is authorized to perform computation in the existing dataset (read)
Main scenario	<i>Data provider (or data consumer) performs a computation through the proxy</i>
Step 1	The data provider (or the data consumer), via a dedicated application, requests to perform a computation in the cloud, over data previously protected
Step 2	The system identifies the dataspace and the outsourced dataset from which the computation service will access data

Step 3	The system controls that the data provider (or the data consumer) is authorized to perform computation (according to the access rights defined in his/her profile and according to his/her authorizations on the dataset)
Step 4	The proxy adapts the computation process query according to protection set initially
Step 5	The proxy orchestrates the computation process, invoking one or several times the computation service provided by the CLARUS-enabled CSP(s). Operations can be parallelized, but the process could be iterative for operations with internal dependencies
Step 6	The computation service provided by the CLARUS-enabled CSP(s) performs the computation(s) over protected data and returns the protected result to the proxy
Step 7	The proxy unprotects and aggregates the intermediate result(s) and composes the final result of the computation
Step 8	The proxy provides the data provider (or the data consumer) with the clear (unprotected) result of the computation
Post-conditions	
[Post-condition 1]	The data provider (or the data consumer), is provided with clear (unprotected) result of the computation
[Post-condition 2]	The clear (unprotected) process query and the clear (unprotected) computation result are not disclosed to the CSP(s) or to any other unauthorized party

Computation directly in the cloud (UC-EUP-10)

UC-EUP-10	<i>Computation directly in the cloud</i>
Actors	End-user
Description	The end-user wants to perform a computation over protected data in the cloud
Pre-conditions	
[Pre-condition 1]	Data have previously been protected and stored in the CSP(s) by the data provider
Main scenario	<i>End-user requests a computation on attributes that have not been protected</i>
Step 1	The end-user performs a computation directly in the cloud, over data attributes that have not been protected
Step 2	The cloud computation service provided by the CLARUS-enabled CSP performs the computation over the unprotected data and provides the end-user with an accurate computation result
Variation 1	<i>End-user requests a computation on attributes that have been protected</i>
Step 1	The end-user performs a computation directly in the cloud, over data attributes that have been protected
Step 2	The cloud computation service provided by the CLARUS-enabled CSP performs the computation over the protected data and provides the end-user with a result that may or may not be accurate (depending on the kind of computation and the parameters of the protection applied to the attributes)
Post-conditions	
[Post-condition 1]	If the end-user has requested a computation on attributes that have not been protected, then the results will be perfectly accurate
[Post-condition 2]	If the end-user has requested a computation on attributes that have been protected, then the result that <i>may or may not be</i> accurate (depending on the kind of computation

	and the parameters of the protection applied to the attributes)
--	---

3.3 Security management

3.3.1 Manage security policies

1.1.1.31 Overview

This scenario describes how the security manager can define and configure the security policy for a CLARUS proxy.

Security policy is managed through the notion of dataspace. The security manager defines the security policy to apply to the dataspace by selecting the data attributes to protect and according the usages on the data. The system will automatically apply the associated security policy on the datasets created by the users. The system determines the dataspace of a dataset thanks to the endpoints used by the applications/services.

System

In this scenario the system is the CLARUS proxy running on trusted local premises.

UML diagram

The UML diagram below represents the corresponding use cases (and actors).

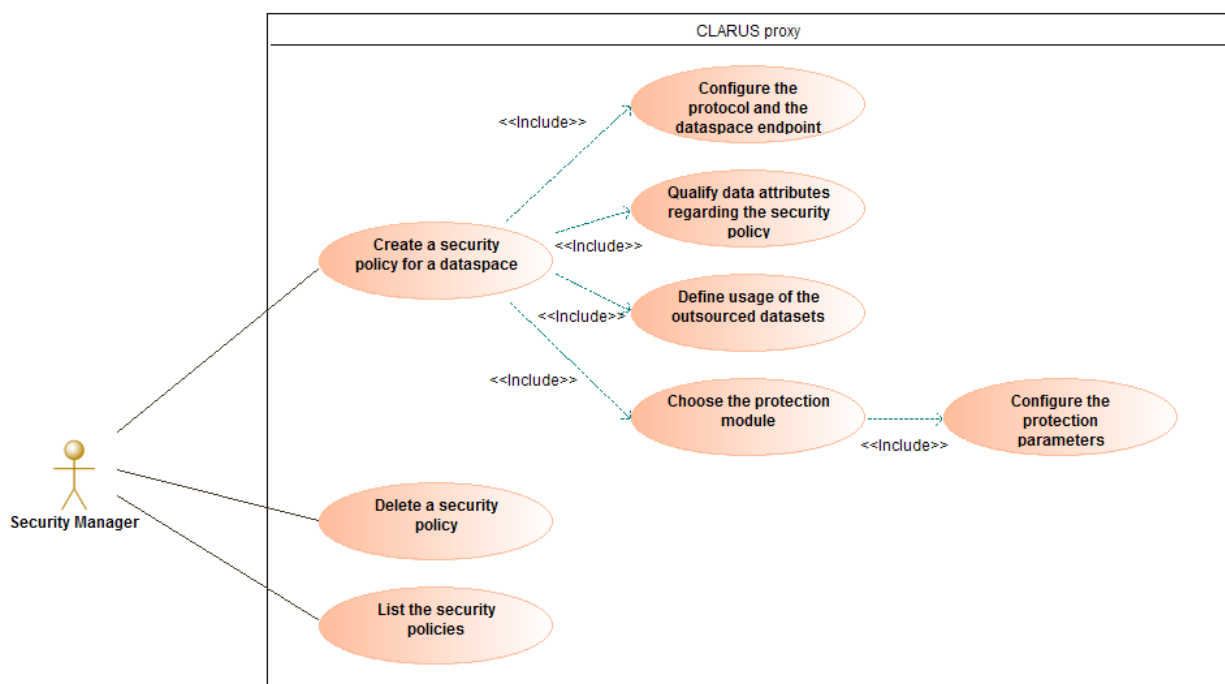


Figure 12 - Security Policy Management Use Case Diagram

1.1.1.32 Use cases

Here again, it is important to emphasize the difference between a dataspace and a dataset :

- a *dataspace* is a virtual space linking applications on premises with cloud storage services. In other terms, a *dataspace* connects to a data source in the cloud. This data source can be a database, a directory, a web features server, etc.
- a *dataset* is the data as such. It can be the rows in a database table, a file in a directory, the content of an XML tree, etc. The *dataset* is associated to the *dataspace* in so far as the *dataspace* defines the connection to the source containing the dataset.

The *dataspace* construct allows connection parameters to be defined once, rather than for each dataset in a source. As such, it is necessary to declare a *dataspace* before uploading datasets within it.

Create a security policy for a dataspace (UC-SEC-01)

UC-SEC-01	<i>Create a security policy for a dataspace</i>
Actors	Security manager
Description	The security manager wants to create a new policy, defining the protocol for access, the usage of outsourced datasets within the <i>dataspace</i> and configuring appropriate security techniques
Pre-conditions	
[Pre-condition 1]	The security manager is authenticated to manage security policies
Main scenario	
Step 1	The security manager inputs the policy name
Step 2	The system returns a security policy identifier (policyid) for further configuration
Step 3	The security manager configures the <i>dataspace</i> endpoint for this security policy (see UC-SEC-02:0)
Step 4	The security manager qualifies data attributes regarding the security policy (see UC-SEC-03:0)
Step 5	The security manager defines the usage of the outsourced datasets (see UC-SEC-04:0)
Step 6	The security manager configures the protection module and protection parameters (see UC-SEC-05:0)
Step 7	The security manager registers the security policy
Step 8	The system creates the new security policy for the corresponding <i>dataspace</i>
Post-conditions	
[Post-condition 1]	A new security policy has been created

Configure the protocol and the dataspace endpoint (UC-SEC-02)

UC-SEC-02	<i>Configure the protocol and the dataspace endpoint</i>
Actors	Security manager
Description	The security manager wants to select and configure the <i>dataspace</i> endpoint (protocol, port, base url) to access the outsourced datasets within it
Pre-conditions	
[Pre-condition 1]	The security manager has started to create a security policy (see UC-SEC-01:0)
[Pre-condition 2]	The supported transfer protocols (e.g. PostgreSQL, OWS, S3, etc.) and their description are available through: • the list of protocols natively supported by CLARUS, • the protocols supported via additional plugins (installed by the IT team)

Main scenario	
Step 0	The security manager selects the security policy under creation
Step 1	The security manager inputs the protocol name
Step 2	The security manager inputs the port
Step 3	The security manager inputs the endpoint base URL
Step 4	The system saves the dataspace endpoint and returns the endpoint URL
Post-conditions	
[Post-condition 1]	The endpoint of the dataspace (to which the security policy will be applied) is configured

Qualify data attributes regarding the security policy (UC-SEC-03)

UC-SEC-03	<i>Qualify data attributes regarding the security policy</i>
Actors	Security manager, cloud storage service(s) (secondary)
Description	The security manager wants to define data attributes regarding the security policy: a data attribute is identified by a path, a type and a data type.
Pre-conditions	
[Pre-condition 1]	The security manager has started to create a security policy (see UC-SEC-01:0)
Main scenario	
Step 0	The security manager selects the security policy under creation
Step 1	The security manager inputs the path of the data attributes to be qualified (e.g. <database>/<table>/<column> for a database column or /<file>/<attribute> for a feature in a file)
Step 2	The security manager defines the corresponding “data type” as either : - categoric - categoric ordinal, - numeric discrete, - numeric continuous, - geometric object, - etc.
Step 3	The security manager defines the corresponding “attribute type” (i.e. qualifies the data attribute regarding security) as either: - non-confidential, - confidential, - identifier, - quasi-identifier
Post-conditions	
[Post-condition 1]	The data attributes are qualified regarding security

Define usage of the outsourced datasets (UC-SEC-04)

UC-SEC-04	<i>Define usage of the outsourced datasets</i>
Actors	Security manager
Description	The security manager selects the usage(s) of the outsourced datasets within a dataspace. The selected usage(s) shall cover one of the following scenarios: • Scenario 1 : Store data in the cloud,

	• Scenario 2 : Search in the cloud, • Scenario 3 : Update in the cloud, • Scenario 4 : Computation in the cloud.
Pre-conditions	
<i>[Pre-condition 1]</i>	The security manager has started to create a security policy (see UC-SEC-01:0)
Main scenario	
Step 0	The security manager selects the security policy under creation
Step 1	The system provides the security manager with the list of possible usages
Step 2	The security manager selects one (or more) usage(s)
Step 3	The system verifies the selected usage(s) are not incompatible with each other(s)
Step 4	The system saves the usage(s) of the outsourced datasets within the dataspace
Post-conditions	
<i>[Post-condition 1]</i>	The usage(s) has been defined for the outsourced datasets within a dataspace

Choose the protection module (UC-SEC-05)

UC-SEC-05	<i>Choose the protection module</i>
Actors	Security manager
Description	The security manager configures the protection modules that are appropriate according to the dataspace configuration and the data usage
Pre-conditions	
<i>[Pre-condition 1]</i>	The security manager has started to create a security policy, defined the dataspace endpoint and qualified the data attributes to protect
<i>[Pre-condition 2]</i>	The security manager has selected the usage of the outsourced datasets
Main scenario	
Step 0	The security manager selects the security policy under creation
Step 1	The system determines the list of protection modules that are applicable according to the dataspace configuration and data usage
Step 2	The system displays the list of possible protection modules to the security manager (highlighting the most suitable module). It can be : - anonymization, - splitting, - encryption, - etc.
Step 3	The security manager chooses one of the appropriate protection module
Step 4	For each protected module selected, the security manager refines the configuration filling the required parameters (see UC-SEC-06:0)
Step 5	The system saves the protection module(s) to be applied to the security policy under creation
Post-conditions	
<i>[Post-condition 1]</i>	The protection techniques with their configuration have been defined

Configure the protection parameters (UC-SEC-06)

UC-SEC-06	<i>Configure the protection parameters</i>
Actors	Security manager
Description	The security manager configures the protection parameters for a given protection module and a given attribute type
Pre-conditions	
[Pre-condition 1]	The security manager has started to create a security policy, defined the dataspace endpoint and qualified the data attributes to protect
[Pre-condition 2]	The security manager has selected the usage of the outsourced datasets
[Pre-condition 3]	The security manager has started to configure the protection module to be applied
Main scenario	
Step 0	The security manager selects the security policy under creation
Step 1	The security manager inputs the attribute type to be further configured, e.g. - non-confidential, - confidential, - identifier, - quasi-identifier
Step 2	Depending on the chosen protection module (see UC-SEC-05:0), the system displays the list of possible protection techniques. e.g. in case of anonymization, it can be <i>suppression</i> , <i>k-anonymity</i> , <i>t-closeness</i> , <i>coarsening</i> , etc.
Step 3	The security manager selects a protection technique
Step 4	According to the protection technique selected, the system displays the list of possible protection parameters, e.g. It can be <i>k</i> for k-anonymity, <i>t</i> for t-closeness, <i>radius</i> for coarsening, etc.
Step 5	The security manager sets a value for each of the protection parameters
Post-conditions	
[Post-condition 1]	The protection techniques with their configuration have been defined

Delete a security policy (UC-SEC-07)

UC-SEC-07	<i>Delete a security policy</i>
Actors	Security manager
Description	The security manager wants to delete a security policy
Pre-conditions	
[Pre-condition 1]	The security manager has created and registered security policies (see UC-SEC-01:0)
Main scenario	
Step 1	The security manager provides the identifier of the security policy to be deleted
Step 2	The system deletes the security policy
Post-conditions	
[Post-condition 1]	The security policy has been deleted

List the security policies (UC-SEC-08)

UC-SEC-08	<i>List the registered security policies</i>
Actors	Security manager

Description	The security manager wants to get a list of the security policies previously created and registered.
Pre-conditions	
[Pre-condition 1]	The security manager has created and registered security policies (see UC-SEC-01:0)
[Pre-condition 2]	The security manager is authenticated to manage security policies
Main scenario	
Step 1	The system provides the security manager with the list of registered security policies
Post-conditions	
[Post-condition 1]	The list of registered security policies is displayed

3.3.2 Manage user access rights

1.1.1.33 Overview

This scenario refers to the actions carried out for the purpose of access rights management. The users have a dedicated profile that includes access rights on the dataspace and on the outsourced datasets.

This scenario considers the internal users of the organization that access to outsourced the datasets through the CLARUS proxy running on trusted local premises.

This scenario also considers external users that want to access outsourced datasets managed by the CLARUS proxy running on trusted local premises. In that case, a technical user is associated to each external CLARUS proxy and accesses outsourced datasets in behalf of one or more external user(s).

System

In this scenario the system is the CLARUS proxy running on trusted local premises.

One CLARUS proxy is involved, but additional CLARUS proxies may be involved.

UML diagrams

The UML diagram below represents the corresponding use cases (and actors).

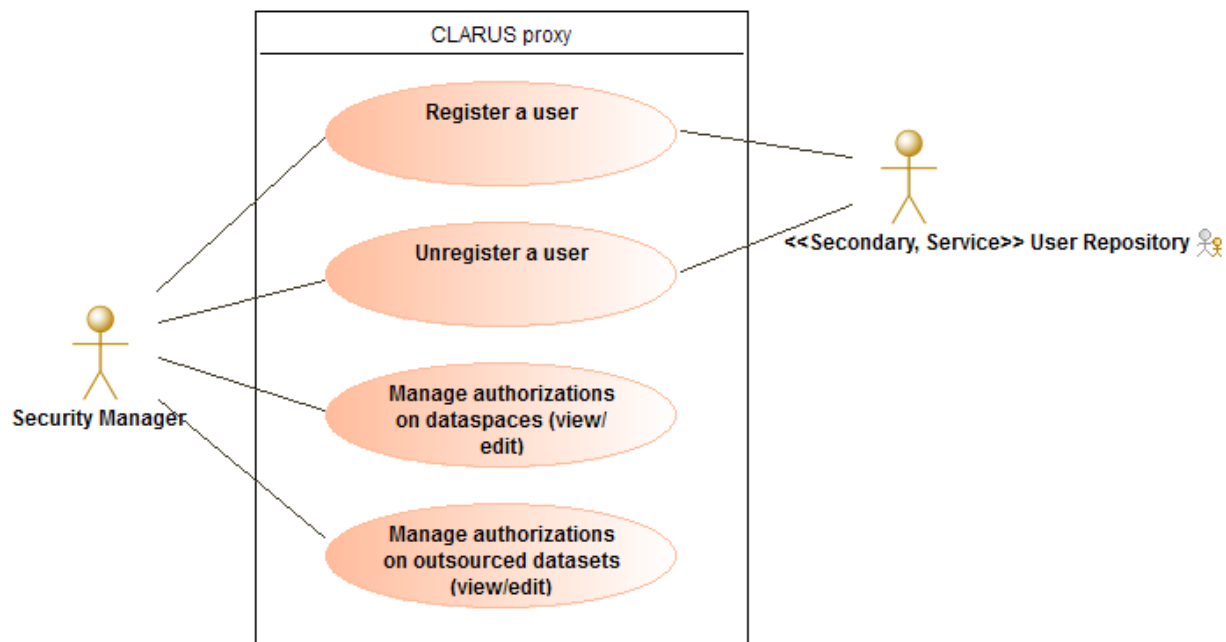


Figure 13 - User Access Rights Management Use Case Diagram

1.1.1.34 Use cases

Register a user (UC-SEC-09)

UC-SEC-09	Register a user
Actors	Security manager, user repository (secondary)
Description	The security manager wants to securely register a user so that the user will be able to access the outsourced dataset(s) (according to its authorizations)
Pre-conditions	
[Pre-condition 1]	The system is configured to access the user repository provided by the IT infrastructure
[Pre-condition 2]	The security manager is authenticated to manage user access rights
Main scenario	<i>Register an internal user</i>
Step 1	The system prompts the security manager to fill user search attributes (e.g. login, mail address, name)
Step 2	The security manager partially or entirely fills the search attributes
Step 3	The system requests the user repository to search the users matching the given attributes
Step 4	The user repository returns the users matching the given attributes
Step 5	The system lists the users matching the given attributes. The system indicates if users are already registered
Step 6	The security manager selects one user to register
Step 7	The system registers the selected user, setting the default profile (no access rights on the existing dataspace(s))
Variation 1	<i>Create a new user (that is not managed by the IT user repository)</i>
Step 1	The system prompts the security manager to fill user attributes (e.g. login)
Step 2	The security manager entirely fills the user attributes

Step 3	The security manager confirms the user creation
Step 4	The system creates a new user, setting the default profile (no access rights on the existing dataspace(s))
Variation 2	<i>Create a new technical user (that identifies an external CLARUS proxy)</i>
Step 1	The system creates a new technical user for an external CLARUS proxy (e.g. identified with the IP address), setting the default profile (no access rights on the existing dataspace(s))
Post-conditions	
[Post-condition 1]	The new registered user has a default profile with no access rights on the existing dataspace(s) and no authorizations on the outsourced dataset(s)
[Post-condition 2]	The external CLARUS proxy identified by a new registered technical user could access the outsourced dataset(s) managed by the system according to the profile of the technical user

Unregister a user (UC-SEC-10)

UC-SEC-10	<i>Unregister a user</i>
Actors	Security manager, user repository (secondary)
Description	The security manager wants to securely unregister a user, deleting his/her profile, so that the user could definitively not access the outsourced dataset(s)
Pre-conditions	
[Pre-condition 1]	The system is configured to access the user repository provided by the IT infrastructure
[Pre-condition 2]	The user has previously been registered
[Pre-condition 3]	The security manager is authenticated to manage user access rights
Main scenario	<i>Unregister an internal user</i>
Step 1	The system lists the registered users
Step 2	The security manager selects the user to unregister and confirms unregistration
Step 3	The system unregisters the selected user, deleting his/her profile (no access rights on the existing dataspace(s) and on the outsourced datasets)
Variation 1	<i>Delete a user (that is not managed by the IT user repository)</i>
Step 4	The system deletes the selected user
Variation 2	<i>Delete a technical user (that represents an external CLARUS proxy)</i>
Step 1	The system deletes the user identified by the IP address, deleting the profile (no access rights on the existing dataspace(s) and on the outsourced datasets)
Post-conditions	
[Post-condition 1]	The user is unregistered and could definitively not access the outsourced dataset(s) managed by the system
[Post-condition 2]	The external CLARUS proxy identified by a technical user that is unregistered could definitively not access the outsourced dataset(s) managed by the system
[Post-condition 3]	The user profile (with access rights on the existing dataspace(s) on the outsourced datasets) is deleted
[Post-condition 4]	The user is deleted if it is not managed by the user repository provided by the IT infrastructure

Manage authorizations on dataspace (view/edit) (UC-SEC-11)

UC-SEC-11	<i>Manage authorisations of users on a dataspace</i>
Actors	Security manager
Description	The security manager wants to securely manage the authorizations (read/write) of the users on the declared dataspace
Pre-conditions	
[Pre-condition 1]	At least one dataspace has been configured
[Pre-condition 2]	The security manager is connected to CLARUS and launches the “Manage access rights” interface
Main scenario	<i>View user authorizations on a dataspace</i>
Step 1	The security manager displays the list of dataspace
Step 2	The security manager selects a dataspace to view the authorizations
Step 3	The system shows the users which have authorizations on the dataspace
Step 4	The security manager selects a user (among the authorized ones)
Step 5	The system shows the authorizations the selected user has on the dataspace: read, write or none
Variation	<i>Edit user authorizations on a dataspace</i>
Step 6	The security manager selects or unselects the read/write authorizations displayed by the system.
Step 7	The security manager confirms the modifications
Step 8	The system saves the modified authorizations
Post-conditions	
[Post-condition 1]	The authorizations of the users on the dataspace are displayed

Manage authorizations on outsourced datasets (view/edit) (UC-SEC-12)

UC-SEC-12	<i>Manage authorisations of users on an outsourced dataset</i>
Actors	Security manager
Description	The security manager wants to securely manage the authorizations (read, update, delete) of the users on an existing outsourced dataset
Pre-conditions	
[Pre-condition 1]	At least one outsourced dataset has been created within a dataspace through the system
[Pre-condition 2]	The security manager is connected to CLARUS and launches the “Manage user permissions on datasets” interface
Main scenario	<i>View authorisations of users on an outsourced dataset</i>
Step 1	The security manager displays the list of dataspace and selects one
Step 2	The system lists the outsourced dataset(s) stored within the selected dataspace
Step 3	The security manager selects an outsourced dataset to view the authorizations
Step 4	The system shows the owner of the outsourced dataset and lists the users which have authorizations on the outsourced dataset
Step 5	The security manager selects a user (among the authorized ones)

Step 6	The system shows the authorizations the selected user has on the outsourced dataset: read, update, delete
Variation	<i>Edit authorisations of users on an outsourced dataset</i>
Step 1	The security manager selects or unselects the read, update and delete authorizations displayed by the system.
Step 2	The security manager confirms the modifications
Step 3	The system saves the modified authorizations
Post-conditions	
[Post-condition 1]	The authorizations of the users on existing outsourced dataset(s) are displayed

3.3.3 Manage multiple proxies

1.1.1.35 Overview

This scenario refers to the actions carried out for the purpose of external CLARUS proxies management. The external CLARUS proxies have a dedicated certificate to connect through a secured channel (e.g. using VPN).

System

The system is the CLARUS proxy running on trusted local premises (local CLARUS proxy).

In this scenario, an external CLARUS proxy is involved (running on premises of the organization or of an external organization).

UML diagram

The UML diagram below represents the corresponding use cases (and actors).

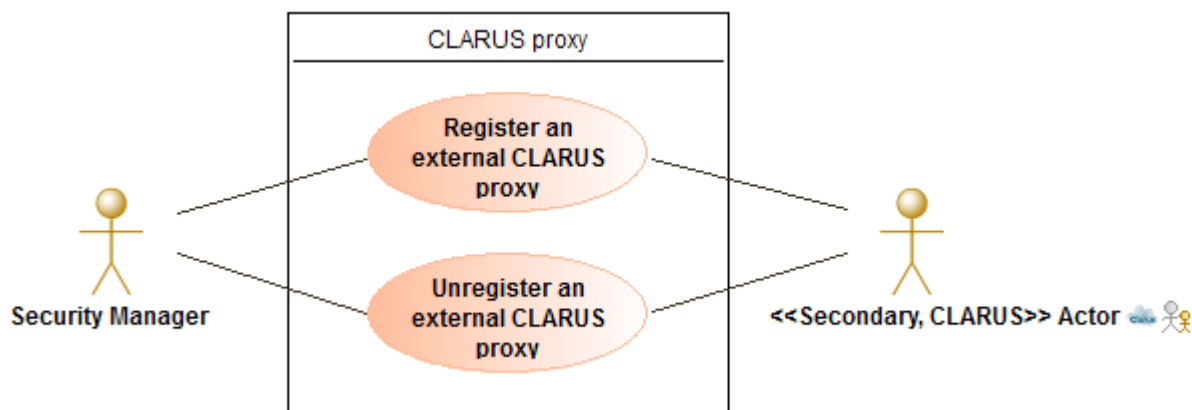


Figure 14 - Multiple Proxies Management Use Case Diagram

1.1.1.36 Use cases

Register an external CLARUS proxy (UC-SEC-13)

UC-SEC-13	<i>Register an external CLARUS proxy</i>
Actors	Security manager of the local CLARUS proxy (system), the external CLARUS proxy
Description	The use case describes the preparations needed to be able to access outsourced dataset from an external CLARUS proxy . Connections are established using a secured

	channel (e.g. using VPN)
Pre-conditions	
[Pre-condition 1]	Both security managers of the two involved CLARUS proxies have prepared the secured channel (e.g. exchanging a VPN certificate)
[Pre-condition 2]	Incoming connection from the external CLARUS proxy must be identifiable (e.g. incoming VPN connection could be mapped to a static internal IP address)
[Pre-condition 3]	The security manager is authenticated to manage multiple proxies
Main scenario	
Step 1	The system prompts the security manager for the digital proof of identity (e.g. the VPN certificate) and for an identifier of the external CLARUS proxy (e.g. the static internal IP address)
Step 2	The security manager gives the digital proof of identity (e.g. the VPN certificate) and the identifier of the external CLARUS proxy (e.g. the static internal IP address)
Step 3	The system registers the digital proof of identity (e.g. the VPN certificate) and stores the identifier of the external CLARUS proxy (e.g. the static internal IP address)
Step 4	The system generates and stores a key for connections from the external CLARUS proxy
Step 5	According to the use case UC-SEC-09, a technical user is registered for the external CLARUS proxy and mapped to the key and to its identifier (e.g. the static internal IP address)
Post-conditions	
[Post-condition 1]	The digital proof of identity (e.g. the VPN certificate) is stored and a technical user for the external CLARUS proxy is registered, which is mapped to its identifier (e.g. the static internal IP address)

Unregister an external CLARUS proxy (UC-SEC-14)

UC-SEC-14	<i>Unregister an external CLARUS proxy</i>
Actors	Security manager of the local CLARUS proxy (system), the external CLARUS proxy
Description	The use case describes the unregistering of an registered external CLARUS proxy
Pre-conditions	
[Pre-condition 1]	The external CLARUS proxy has been previously registered
[Pre-condition 2]	The security manager is authenticated to manage multiple proxies
Main scenario	
Step 1	The system lists the registered external CLARUS proxies
Step 2	The security manager selects the external CLARUS proxy to unregister and confirms unregistration
Step 3	The system removes the key associated to the external CLARUS proxy .
Step 4	The system unregisters the technical user that identify the external CLARUS proxy (use case UC-SEC-10)
Step 5	The secured channel (e.g. a VPN connection) is removed, the digital proof of identity (e.g. the VPN certificate) is deleted
Post-conditions	
[Post-condition 1]	The secured channel (e.g. a VPN connection) is deactivated by removing the digital proof of identity (e.g. the VPN certificate) and the technical user that identify the

	external CLARUS proxy and the according key are removed
--	---

3.4 Monitoring

3.4.1 Standard monitoring

1.1.1.37 Overview

This scenario refers to the actions for enabling standard monitoring of CLARUS proxy.

System

In this scenario the system encompasses:

- The CLARUS proxy running on trusted local premises,
- The CLARUS adopting CSP.

One CLARUS proxy is involved. Additional CLARUS proxies may be involved, and monitoring should correlate the information provided by distributed CLARUS proxies.

UML diagram

The UML diagram below represents the corresponding use case (and actors).

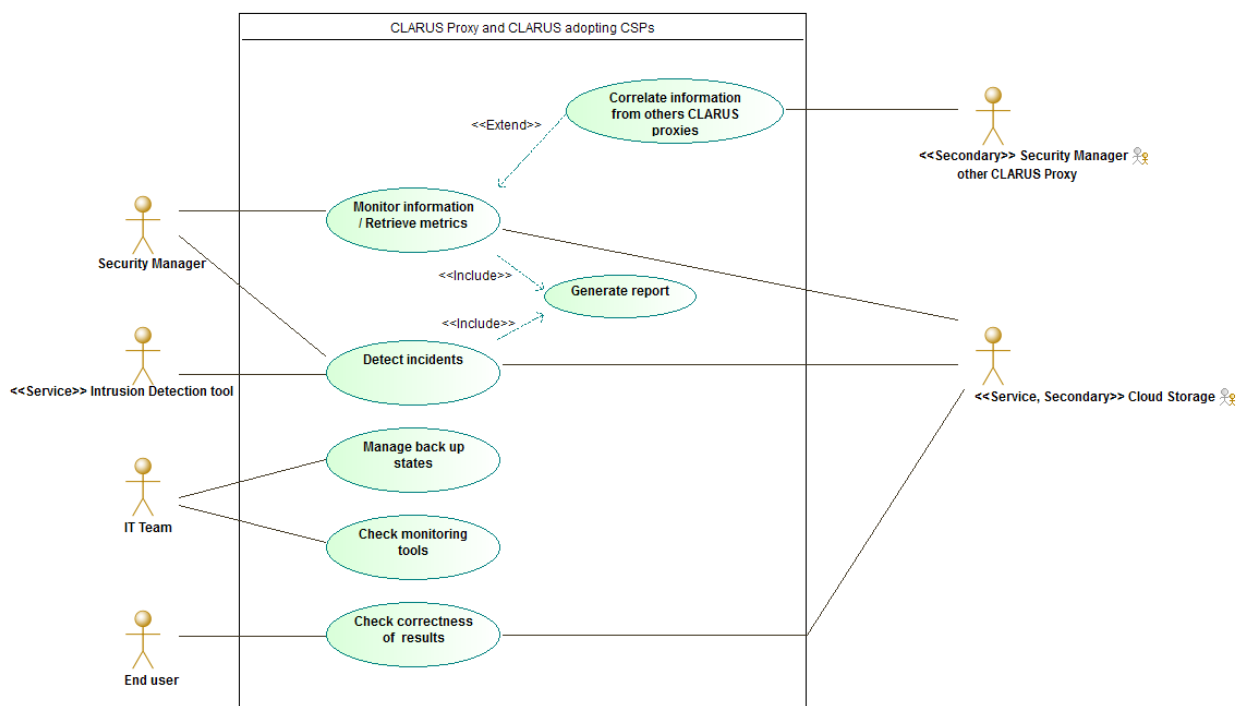


Figure 15 - Standard Monitoring Use Case Diagram

1.1.1.38 Use cases

Monitor information / Retrieve metrics (UC-MON-01)

UC-MON-01	Monitor information / Retrieve metrics
Actors	Security manager
Description	The Security Manager wants to supervise the standard use of CLARUS features in order

	to detect potential incidents
Pre-conditions	
[Pre-condition 1]	Monitoring tool installed in the system
[Pre-condition 2]	A risk analysis is already performed to define sensitive data to protect and security and business requirements. It also defines different risks and vulnerabilities of the proposed CLARUS solution.
[Pre-condition 3]	List of metrics and indicators to be monitored
Main scenario	<i>Periodical monitoring of the system status</i>
Step 1	The Security Manager connects to the monitoring interface
Step 2	The Security Manager selects the list of relevant reports (including business and security metrics) that he/she needs to monitor
Step 3	The monitoring tool provides the values of the requested metrics (as a set of graphs and tables) in a web-based report
Step 4	The Security Manager checks if there is any security incident and acts in consequence (See UC-MON-02 : o)
Post-conditions	
[Post-condition 1]	The monitoring informations and relevant metrics retrieved are synthetised in a web-based reporting output.

Detect incidents (UC-MON-02)

UC-MON-02	<i>Detect incidents</i>
Actors	Security Manager
Description	The Security Manager supervising the standard use of CLARUS features detects incidents via the monitoring tools
Pre-conditions	
[Pre-condition 1]	List of security incidents to be detected already defined, as well as rules to detect those incidents from monitored information
[Pre-condition 2]	The monitoring tool continuously supervise the system at different strategic observation points (including the client device, the proxy, and the cloud provider)
Main scenario	<i>Security incident detection (manual response)</i>
Step 1	The monitoring tool detects a security incident
Step 2	The Security Manager is notified of the detected security incident (e.g., via the Web report)
Step 3	The Security Manager decides to apply a counter-measure to mitigate the detected security incident
Step 4	The Security Manager checks the impact of the security incident
Variation 1	<i>Security incident detection (automatic response)</i>
Step 2	The monitoring tool automatically executes the necessary counter-measures to mitigate the security incident
Step 3	The monitoring tool notifies the Security Manager about the security incident occurrence and the counter-measures taken
Step 4	The Security Manager checks the impact of the security incident
Post-conditions	

<i>[Post-condition 1]</i>	The detected incidents and taken counter-measures are synthetised in a web-based reporting output
---------------------------	---

Manage backup states (UC-MON-03)

UC-MON-03	<i>Manage backup states</i>
Actors	IT Team
Description	The IT Team handle some backup states in order to prevent incidents as CLARUS failures, requiring to configure or restore a new CLARUS proxy
Pre-conditions	
<i>[Pre-condition 1]</i>	The IT Team provides a backup state
Main scenario	<i>Restore a previous backup state or configure the CLARUS proxy</i>
Step 1	The IT Team connects to the CLARUS proxy
Step 2	The IT Team restores/reconfigures the CLARUS proxy using a previous backup state
Variation 1	<i>Make a periodical proxy configuration backup</i>
Step 2	The IT Team copies the CLARUS proxy configuration to create a backup state
Post-conditions	
<i>[Post-condition 1]</i>	A new CLARUS proxy is reconfigured or initial CLARUS proxy is restored after an incident has been detected

Check monitoring tools (UC-MON-04)

UC-MON-04	<i>Check monitoring tools</i>
Actors	IT Team
Description	The IT Team can check the functionality and performance of CLARUS monitoring tools in a dedicated integration environment
Pre-conditions	
<i>[Pre-condition 1]</i>	Monitoring tools already installed and configured in the system
Main scenario	<i>Functionality and performance checking of security tools</i>
Step 1	The IT Team connects to the monitoring tools management environment
Step 2	The IT Team checks the functionality of the monitoring tools
Step 3	The IT Team checks the performance of the monitoring tools
Step 4	The IT Team provides a report on the functionality and performance status of the monitoring tools
Post-conditions	
<i>[Post-condition 1]</i>	The functionality and performance of CLARUS monitoring tools has been tested in a dedicated integration environment

Check the correctness of results (UC-MON-05)

UC-MON-05	<i>Check the correctness of results</i>
Actors	End user, IT team
Description	The end-user checks that the data storage/search/update /computation has been

	correctly performed
Pre-conditions	
[Pre-condition 1]	The end-user is authorised to access the system
[Pre-condition 2]	The end-user has performed a data storage/search/update/computation
Main scenario	<i>The end-user verifies the correctness of results</i>
Step 1	The end-user verifies that the data storage/search/update/computation has been correctly performed
Variation 1	<i>An error detected</i>
Step 1	If an error in data storage/search/update/computation is detected, the end user notifies the IT Team to solve the issue
Post-conditions	
[Post-condition 1]	The end user has checked (and notified the IT Team if an error has been detected) the correctness of storage/search/update/computation results

Correlate information from other CLARUS proxies (UC-MON-06)

UC-MON-06	Correlate information from distributed CLARUS proxies
Actors	Security Manager , Security Manager from other CLARUS proxy
Description	The Security Manager performs information correlation between two (or more) distributed CLARUS proxies
Pre-conditions	
[Pre-condition 1]	The Security Manager is authorised to get information coming from distributed CLARUS proxies
[Pre-condition 2]	The information coming from the proxies needs to be in compatible format to allow direct correlation
Main scenario	<i>Correlation of information coming from distributed CLARUS proxies (e.g., proxies 1 and 2)</i>
Step 1	The Security Manager of proxy 1 requests the information from proxy 2
Step 2	The monitoring tool installed in proxy 1 performs the correlation
Step 3	The monitoring tool from proxy 1 provides a web-based report including the correlated information from proxies 1 and 2
Variation 1	<i>Correlation coming from n distributed proxies</i>
Step 1	The Security Manager of proxy 1 requests the information from proxies 2 to n
Step 2	The monitoring tool installed in proxy 1 performs the correlation
Step 3	The monitoring tool from proxy 1 provides a web-based report including the correlated information from proxies 2 to n
Post-conditions	
[Post-condition 1]	The correlated information from distributed proxies is synthetised in a web-based reporting output.

3.4.2 Attack monitoring

1.1.1.39 Overview

This scenario refers to the actions for enabling monitoring of CLARUS proxy in the context of an attack.

System

In this scenario the system encompasses:

- The CLARUS proxy running on trusted local premises,
- The CLARUS adopting CSP.

One CLARUS proxy is involved. Additional CLARUS proxies may be involved, and monitoring should correlate the information provided by distributed CLARUS proxies.

UML diagram

The UML diagram below represents the corresponding use case (and actors).

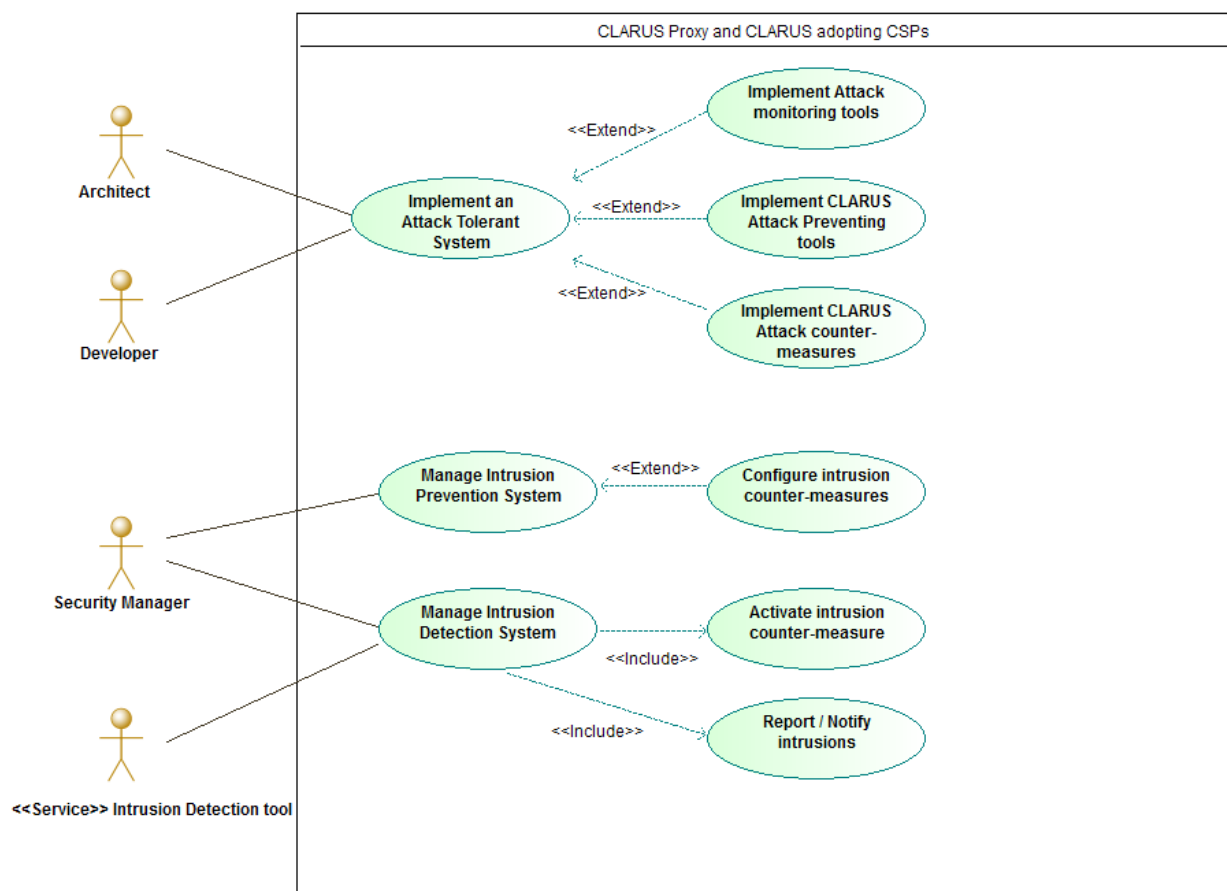


Figure 16 - Attack Monitoring Use Case Diagram

1.1.1.40 Use cases

Implement an Attack Tolerant System (UC-MON-07)

UC-MON-07	<i>Implement an Attack Tolerant System</i>
Actors	Architect ,Developer

Description	The architect and developer wants to develop intrusion management tools in an “Attack Tolerant System “ perspective
Pre-conditions	
[Pre-condition 1]	Implementation of Attack Monitoring tools
[Pre-condition 2]	Implementation of CLARUS Attack Preventing tools
[Pre-condition 3]	Implementation of CLARUS Attack counter-measures
Main scenario	<i>Implementation of an Attack Tolerant System</i>
Step 1	The Architect , considering the Attack Monitoring tools, the CLARUS Attack Preventing tools, and the CLARUS Attack counter-measures designs the Attack Tolerant System to guarantee the system operation even in the presence of an attack
Step 2	The Developer , considering the Attack Monitoring tools, the CLARUS Attack Preventing tools, and the CLARUS Attack counter-measures implements the Attack Tolerant System to guarantee the system operation even in the presence of an attack
Variation 1	<i>Mitigation mechanisms</i>
Step 1	The Architect/Developer implements mitigation mechanisms in case the system needs additional measures if an attack compromises the system
Post-conditions	
[Post-condition 1]	CLARUS system continues running and delivers the required services under the event of an attack.

Manage Intrusion Prevention System (UC-MON-08)

UC-MON-08	<i>Manage Intrusion Prevention System</i>
Actors	Security Manager
Description	The Security Manager handle the Intrusion Prevention monitoring tools in order to provide relevant countermeasures in case of intrusion detection
Pre-conditions	
[Pre-condition 1]	Intrusion prevention tools are installed, configured and running
[Pre-condition 2]	List of intrusions / attacks
[Pre-condition 3]	List of counter-measures
Main scenario	<i>Manage Intrusion Prevention System</i>
Step 1	The Security Manager connects to the Intrusion Prevention System
Step 2	The Security Manager analyses the list of intrusions/attacks and counter-measures to establish a correspondence between intrusions/attacks and counter-measures
Step 3	The Security Manager configures the system to execute the necessary counter-measures in presence of an intrusion/attack
Post-conditions	
[Post-condition 1]	The preventive measures, as the configuration of intrusion countermeasures, have been taken

Manage Intrusion Detection System (UC-MON-09)

UC-MON-09	<i>Manage Intrusion Detection System</i>
------------------	--

Actors	Security Manager
Description	The Security Manager handles the Intrusion Detection System, after an intrusion has been detected
Pre-conditions	
[Pre-condition 1]	Intrusion detection tools are installed, configured and running
[Pre-condition 2]	List of intrusions/attacks
[Pre-condition 3]	List of counter-measures
Main scenario	<i>Manage Attack Detection System (manual mode)</i>
Step 1	The Security Manager is notified on real-time via web-based reporting of the occurrence of an intrusion/attack in the system
Step 2	The Security Manager manually activates the corresponding counter-measures to mitigate the effects of the detected intrusion/attack
Step 3	A web-based report is created detailing the detected intrusion/attack and the counter-measures taken
Variation 1	<i>Manage Intrusion Detection System (automatic mode)</i>
Step 0	The Intrusion Detection Tool detects an attack
Step 1	The Intrusion Detection Tool automatically executes the necessary counter-measures to mitigate the effects of the detected intrusion/attack
Step 2	A web-based report is created detailing the detected intrusion/attack and the counter-measures taken
Post-conditions	
[Post-condition 1]	The dedicated countermeasures has been launched after an intrusion/attack has been detected. An associated notification has been generated as a web-based reporting output.

3.5 Administration

3.5.1 CLARUS proxy installation

1.1.1.41 Overview

This scenario describes the actions required for installing the CLARUS proxy.

System

In this scenario the system refers to the IT system on premises, i.e. in a server within the user's domain (trusted zone). The CLARUS proxy may also be deployed within the client computer, or in any other location trusted by the user.

The CLARUS proxy includes, among others, the monitoring solution to analyse different business and security indicators and is composed of the necessary monitoring, and attack prevention and detection tools.

UML diagram

The UML diagram below represents the corresponding use case (and their actors):

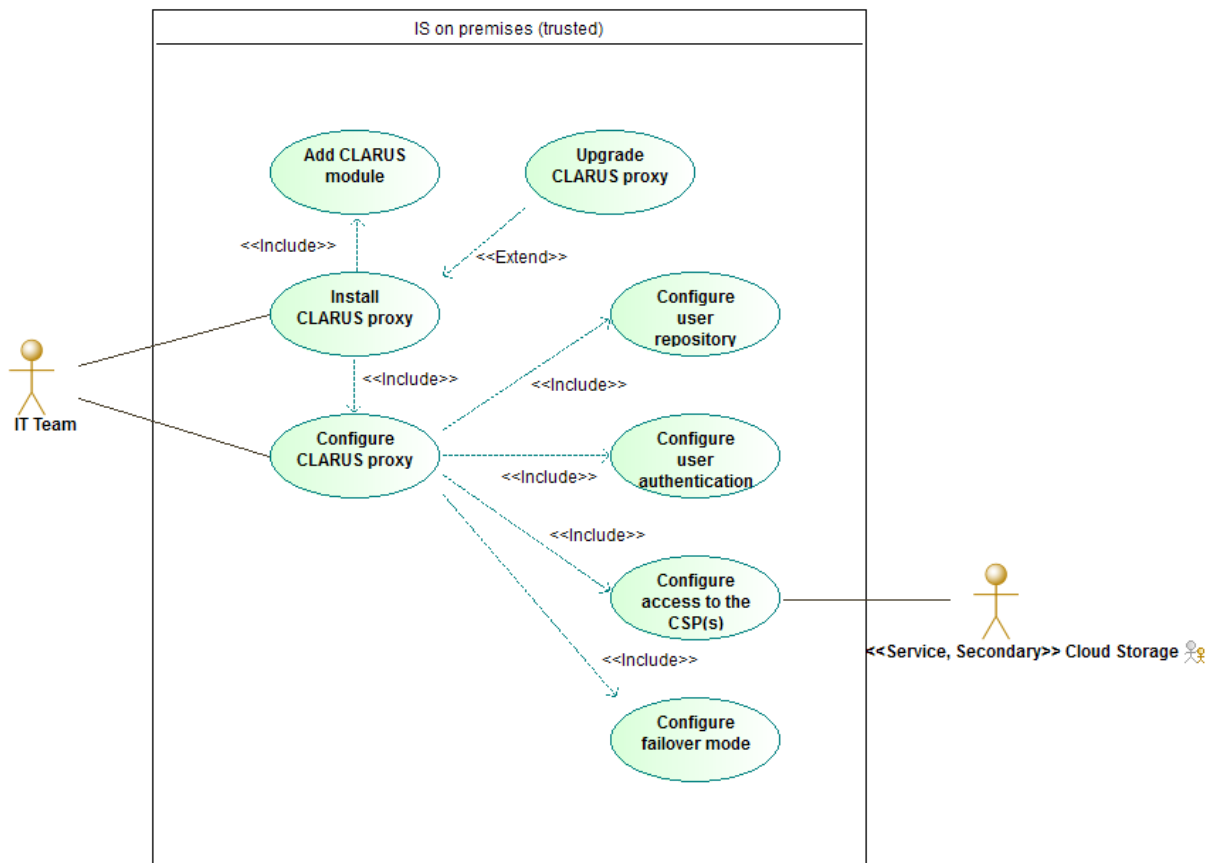


Figure 17 – CLARUS Proxy Installation Use Case Diagram

1.1.1.42 Use cases

Install CLARUS proxy (UC-ADM-01)

UC-ADM-01	Install CLARUS proxy
Actors	IT team
Description	The IT team wants to install a CLARUS proxy on a dedicated server in the local (trusted) IT system
Pre-conditions	
[Pre-condition 1]	No CLARUS proxy is installed yet
[Pre-condition 2]	A CLARUS proxy installation package is available for the target operation system
Main scenario	<i>Install</i>
Step 1	The IT team gets the installation guide of CLARUS proxy in order to follow its instructions
Step 2	The member of the IT team in charge of the installation opens an administrator session on the dedicated server
Step 3	The system controls the access rights of the IT team on the server
Step 4	The member of the IT team launches a command that automatically retrieves the package for CLARUS proxy, installs it on the server and starts it
Variation 1	<i>Add CLARUS module (see UC-ADM-08 : 0)</i>
Post-conditions	

<i>[Post-condition 1]</i>	The CLARUS proxy is installed and is running on the server
---------------------------	--

Upgrade CLARUS proxy (UC-ADM-02)

UC-ADM-02	<i>Upgrade CLARUS proxy</i>
Actors	IT team
Description	The IT team wants to upgrade a CLARUS proxy on a dedicated server in the local (trusted) IT system
Pre-conditions	
<i>[Pre-condition 1]</i>	An obsolete version of CLARUS proxy is installed
<i>[Pre-condition 2]</i>	A new 'CLARUS proxy' package (and its installation guide) is available for the target operation system
Main scenario	<i>Upgrade</i>
Step 1	The member of the IT team in charge of the upgrade opens an administrator session on the dedicated server, and follows the upgrade instructions from the installation guide of the version to install.
Step 2	The system controls the access rights of the IT team on the server
Step 3	The member of the IT team launches a command that stops the CLARUS proxy
Step 4	The member of the IT team launches a command, specifying the CLARUS version as argument, that upgrades CLARUS to the corresponding version and starts it
Post-conditions	
<i>[Post-condition 1]</i>	The CLARUS proxy is upgraded and is running on the server
<i>[Post-condition 2]</i>	Configuration data generated by CLARUS (registered users, outsourced datasets, authorizations, appropriate materials, etc.) are preserved

Configure CLARUS proxy (UC-ADM-03)

UC-ADM-03	<i>Configure CLARUS proxy</i>
Actors	IT team
Description	The IT team wants to configure a CLARUS proxy on the local (trusted) IT system
Pre-conditions	
<i>[Pre-condition 1]</i>	A CLARUS proxy, in its last version, has been installed in the local (trusted) IT system (see UC-ADM-01: 0 or UC-ADM-02: 0)
Main scenario	<i>Configure CLARUS</i>
Step 1	The IT team gets the configuration manual of CLARUS proxy corresponding to the installed version
Step 2	The member of the IT team in charge of the configuration opens an administrator session on the server where the CLARUS proxy is running
Step 3	The system controls the access rights of the IT team on the server
Step 4	The member of the IT team configures the user repository (see UC-ADM-04: 0)
Step 5	The member of the IT team configures the user authentication (see UC-ADM-05: 0)
Step 6	The member of the IT team configures the access to the CSP(s) (see UC-ADM-06: 0)
Variation 1	<i>Configure failover mode (see UC-ADM-07: 0)</i>
Post-conditions	

<i>[Post-condition 1]</i>	The CLARUS proxy is configured
---------------------------	--------------------------------

Configure user repository (UC-ADM-04)

UC-ADM-04	<i>Configure user repository</i>
Actors	IT team
Description	The IT team wants to configure the CLARUS proxy on the local (trusted) information system to connect to a user repository for the management of users' access rights : - Either the IT user repository delivered by the organization (e.g. company LDAP) to which the CLARUS proxy must integrate if possible to search for users and their information (<i>main scenario</i>) - Or an embedded user repository (<i>alternate scenario, e.g., for CLARUS use outside the organization, CLARUS testing purpose</i>).
Pre-conditions	
<i>[Pre-condition 1]</i>	The <i>[Pre-condition 1]</i> and 'Step 1' to '3' described 'UC-ADM-03' are fulfilled.
<i>[Pre-condition 2]</i>	In case of an existing IT user repository, it must be configured to accept queries from the CLARUS proxy
Main scenario	<i>Configure to use the IT user repository</i>
Step 1	The member of the IT team sets and saves the access parameters of the IT user repository that will be used to search for users and their information
Step 2	The CLARUS proxy checks that the IT user repository is accessible
Variation 1	<i>Configure to use the embedded user repository (no IT user repository)</i>
Step 1	The member of the IT team sets and saves the parameters to use an embedded user repository
Step 2	The CLARUS proxy checks that the embedded user repository is accessible
Post-conditions	
<i>[Post-condition 1]</i>	The CLARUS proxy is configured to use the user repository made available (either IT user repository or an embedded user repository)

Configure user authentication (UC-ADM-05)

UC-ADM-05	<i>Configure user authentication</i>
Actors	IT team
Description	The IT team wants to configure the authentication mechanism of the CLARUS proxy running on the local (trusted) IT system
Pre-conditions	
<i>[Pre-condition 1]</i>	The <i>[Pre-condition 1]</i> and 'Step 1' to '3' described 'UC-ADM-03' are fulfilled.
<i>[Pre-condition 2]</i>	If a CLARUS proxy has been configured to use the embedded user repository (see UC-ADM-04: o), it must be configured to use the embedded authentication mechanism (<i>alternate scenario</i>)
Main scenario	<i>Configure to use the IT authentication service</i>
Step 1	The member of the IT team sets and saves the access parameters of the IT authentication service (e.g. company SSO) that will be used to authenticate the users which access the CLARUS proxy

Step 2	The CLARUS proxy checks that the IT authentication service is accessible
Variation 1	<i>Configure to use the embedded authentication mechanism</i>
Step 1	The member of the IT team sets and saves the parameters to use the embedded authentication mechanism
Step 2	The member of the IT team launches a command that restarts the CLARUS proxy
Post-conditions	
[Post-condition 1]	The CLARUS proxy is configured to use either the IT authentication service or the embedded authentication mechanism

Configure access to the CSP(s) (UC-ADM-06)

UC-ADM-06	<i>Configure access to the CSP(s)</i>
Actors	IT team, cloud storage service (secondary)
Description	The IT team wants to configure the access to the cloud storage service of one (or more) CSP(s) towards which the end-users will be directed
Pre-conditions	
[Pre-condition 1]	The [Pre-condition 1] and 'Step 1' to '3' described 'UC-ADM-03' are fulfilled.
[Pre-condition 2]	For each CSP, the IT team has the cloud storage endpoint (URL) and credentials
Main scenario	<i>Configure access to the CSP(s)</i>
Step 1	The member of the IT team sets and saves the cloud storage endpoints (URLs) and credentials of one (or more) cloud storage service(s)
Step 2	The member of the IT team launches a command that restarts the CLARUS proxy
Step 3	The CLARUS proxy checks that each cloud storage service is accessible
Post-conditions	
[Post-condition 1]	The CLARUS proxy is configured to access the cloud storage service of one (or more) CSP(s) towards which the end-users will be directed.

Configure failover mode (UC-ADM-07)

UC-ADM-07	<i>Configure failover mode</i>
Actors	IT team
Description	The IT team wants to configure the CLARUS proxy running on the local (trusted) information system to be part of a failover cluster of CLARUS proxies
Pre-conditions	
[Pre-condition 1]	The [Pre-condition 1] and 'Step 1' described 'UC-ADM-03' are fulfilled <u>for several CLARUS proxies</u> .
[Pre-condition 2]	The CLARUS proxies has been configured to use the IT user repository
Main scenario	<i>Activate failover mode</i>
Step 1	The member of the IT team in charge of the CLARUS configuration opens an administrator session on each server where a CLARUS proxy is running
Step 2	On each server, the system controls the access rights of the IT team on the server
Step 3	On each server, the member of the IT team sets and saves the parameters so that the CLARUS proxies are part of the same failover cluster of CLARUS proxies

Step 4	On each server, the member of the IT team launches a command that restarts the CLARUS proxy
Step 5	On each server, the CLARUS proxy checks that it is part of a failover cluster of CLARUS proxies
Variation 1	<i>Deactivate failover mode</i>
Step 4	The member of the IT team sets and saves the parameters so that the CLARUS proxy is not part of any failover cluster of CLARUS proxies
Step 5	The member of the IT team launches a command that restarts the CLARUS proxy
Post-conditions	
[Post-condition 1]	If failover mode is activated, several CLARUS proxies are part of the same failover cluster of CLARUS proxies
[Post-condition 2]	If failover mode is activated, the CLARUS proxies that are part of the same failover cluster are accessible as a single CLARUS proxy
[Post-condition 3]	If failover mode is deactivated, the CLARUS proxy is not part of any failover cluster of CLARUS proxies

Add CLARUS module (UC-ADM-08)

UC-ADM-08	<i>Add CLARUS module</i>
Actors	IT team
Description	The IT team wants to install one (or more) CLARUS module on the running CLARUS proxy, in order to support features not natively supported
Pre-conditions	
[Pre-condition 1]	A CLARUS proxy has been installed and configured on the IT system (e.g. a server in the users domain) by the IT team (see UC-ADM-01 to -07 :0)
[Pre-condition 1]	An additional CLARUS module has been developed (see UC-IMPL-01: 0)
Main scenario	<i>Add CLARUS module</i>
Step 1	The member of the IT team in charge of the installation opens an administrator session on the server where the CLARUS proxy is running
Step 2	The system controls the access rights of the IT team on the server
Step 3	The member of the IT team downloads the selected module on the server where the CLARUS proxy is running
Step 4	The member of the IT team updates the CLARUS proxy configuration to take the new module into account (e.g. copy in a dedicated folder)
Post-conditions	
[Post-condition 1]	The CLARUS proxy supports the new features brought by the add-on module

3.5.2 CSP service-preserving add-ons installation

1.1.1.43 Overview

This scenario describes the actions required by a CSP (or the IT team on behalf of the CSP) for the integration of CLARUS in the cloud platform in order to provide the end-users with security-enhanced and trust-enhanced cloud services.

System

In this scenario the system is the information system on CSP.

UML diagram

The UML diagram below represents the corresponding use case (and main actor):

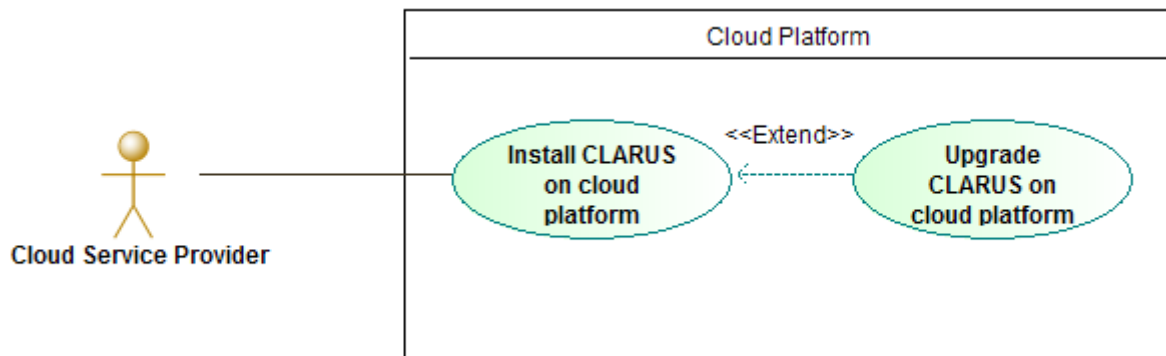


Figure 18 – CSP service-preserving add-ons installation Use Case Diagram

1.1.1.44 Use cases

Install CLARUS on cloud platform (UC-ADM-09)

UC-ADM-09	Install CLARUS on cloud platform
Actors	Cloud service provider
Description	The cloud service provider wants to integrate CLARUS in the cloud platform
Pre-conditions	
[Pre-condition 1]	No CLARUS software is installed yet
[Pre-condition 2]	Installation package(s) of CLARUS components is(are) available to enhance security and trust of cloud services
Main scenario	Install CLARUS components
Step 1	The cloud service provider gets the CSP installation guide of CLARUS in order to follow its instructions
Step 2	The cloud service provider installs one (or more) CLARUS component(s) (e.g. tool(s), service(s), library(ies))
Post-conditions	
[Post-condition 1]	The cloud platform provides the end-users with security-enhanced and trust-enhanced cloud services, in a transparent way.

Upgrade CLARUS on cloud platform (UC-ADM-10)

UC-ADM-10	Upgrade CLARUS on cloud platform
Actors	Cloud service provider
Description	The cloud service provider wants to upgrade the CLARUS components installed on the cloud platform
Pre-conditions	
[Pre-condition 1]	Obsolete versions of CLARUS components are installed

<i>[Pre-condition 2]</i>	New installation package(s) of CLARUS components is(are) available
Main scenario	<i>Upgrade CLARUS components</i>
Step 1	The cloud service provider gets the CSP installation guide of CLARUS corresponding to the version to install in order to follow its upgrade instructions
Step 2	The cloud service provider upgrades one (or more) CLARUS component(s) (e.g. tool(s), service(s), library(ies))
Variation 1	<i>Adapt cloud service(s) (see UC-IMPL-02:0)</i>
Post-conditions	
<i>[Post-condition 1]</i>	The cloud platform provides the end-users with upgraded CLARUS security services

3.6 Development

3.6.1 Develop CLARUS extensions /plugins

1.1.1.45 Overview

This scenario describes the actions required for developing extensions/plugins in order to allow the CLARUS proxy to offer features not natively supported.

System

In this scenario the system refers to the public code repository in which the open source CLARUS software is hosted.

UML diagram

The UML diagram below represents the corresponding use cases (and their actors):

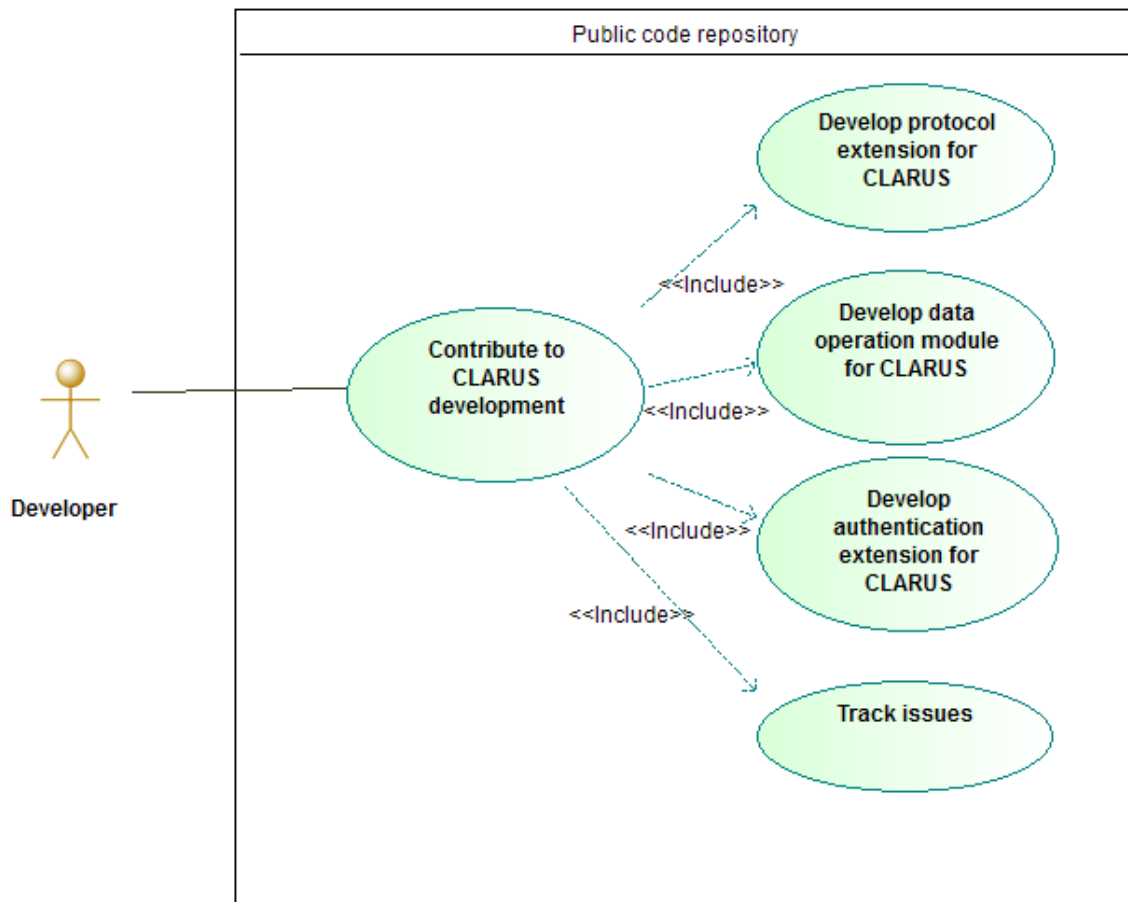


Figure 19 – Develop CLARUS extensions/plugins Use Case Diagram

1.1.1.46 Use cases

Contribute to CLARUS development (UC-DEV-01)

UC-DEV-01	<i>Contribute to CLARUS development</i>
Actors	Developer
Description	The Developer wants to contribute to the CLARUS source code
Pre-conditions	<i>None</i>
Main scenario	<i>Contribute to CLARUS development</i>
Step 1	The Developer joins the CLARUS developer community signing up to the public CLARUS source code repository
Step 2	The Developer gets a local copy of the CLARUS source code repository
Step 3	By following the developer guide and the CLARUS API documentation, the Developer develops new features or enhancements (e.g. support for new protocols, new data protection techniques, new authentication mechanisms, bug fixing, etc.)
Step 4	The Developer commits his/her changes and test that they do not introduce regressions
Step 5	The Developer updates the public CLARUS source code repository
Post-conditions	
[Post-condition 1]	The Developer is member of the CLARUS developer community
[Post-condition 2]	The Developer has contributed to the CLARUS source code

Develop protocol extension for CLARUS (UC-DEV-02)

UC-DEV-02	<i>Develop protocol extension for CLARUS</i>
Actors	Developer
Description	The Developer wants to implement a protocol extension in order to allow the CLARUS proxy to support data transfer protocols not yet supported
Pre-conditions	
[Pre-condition 1]	The Developer is member of the CLARUS developer community
[Pre-condition 2]	The Developer has got a local copy of the CLARUS source code repository
Main scenario	<i>Develop a protocol plugin</i>
Step 1	By following the developer guide and the CLARUS API documentation, the Developer contributes to the development of a CLARUS plugin for a new data transfer protocol
Step 2	The Developer commits his/her changes and test that they do not introduce regressions
Step 3	The Developer updates the public CLARUS source code repository
Post-conditions	
[Post-condition 1]	The Developer has contributed to the development of a new CLARUS protocol plugin

Develop data operation extension for CLARUS (UC-DEV-03)

UC-DEV-03	<i>Develop data operation module for CLARUS</i>
Actors	Developer
Description	The Developer wants to implement a data operation module in order to allow the CLARUS proxy to support a data protection technique not yet supported
Pre-conditions	
[Pre-condition 1]	The Developer is member of the CLARUS developer community
[Pre-condition 2]	The Developer has got a local copy of the CLARUS source code repository
Main scenario	<i>Develop a data operation module</i>
Step 1	By following the developer guide and the CLARUS API documentation, the Developer contributes to the development of a CLARUS module for a new data protection technique
Step 2	The Developer commits his/her changes and test that they do not introduce regressions
Step 3	The Developer updates the public CLARUS source code repository
Post-conditions	
[Post-condition 1]	The Developer has contributed to the development of a new CLARUS data operation module

Develop authentication extension for CLARUS (UC-DEV-04)

UC-DEV-04	<i>Develop authentication extension for CLARUS</i>
Actors	Developer
Description	The Developer wants to implement an extension in order to allow the CLARUS proxy to support authentication methods not yet supported
Pre-conditions	
[Pre-condition 1]	The Developer is member of the CLARUS developer community
[Pre-condition 2]	The Developer has got a local copy of the CLARUS source code repository
Main scenario	<i>Develop an extension for authentication</i>
Step 1	Following the developer guide and the CLARUS API documentation, the Developer contributes to the development of a CLARUS extension for a new authentication mechanism
Step 2	The Developer commits his/her changes and test that they do not introduce regressions

Step 3	The Developer updates the public CLARUS source code repository
Post-conditions	
[Post-condition 1]	The Developer has contributed to the development of a new CLARUS authentication extension

Track issues (UC-DEV-05)

UC-DEV-05	<i>Track issues</i>
Actors	Developer
Description	The Developer wants to use an issue tracker to view all bug reports and add some
Pre-conditions	<i>A bug has been detected</i>
Main scenario	<i>File a bug in the issue tracker</i>
Step 1	The Developer connects to the CLARUS issue tracker
Step 2	The Developer displays the list of reported bugs
Step 3	The Developer checks the bug is not in the list
Step 4	The Developer files a new bug using the issue tracker
Variation 1	<i>Resolve an issue</i>
Step 3	The Developer selects the issue he/she has corrected
Step 4	The Developer updates the status of the issue to closed
Post-conditions	
[Post-condition 1]	All the identified bugs are documented and classified as new or closed

3.6.2 Implement CLARUS module for the cloud

1.1.1.47 Overview

This scenario describes the actions required by a CSP (or the IT team on behalf of the CSP) for modifying the implementation of a cloud service in order to enhance security and trust capabilities.

System

In this scenario the system is the information system on CSP.

UML diagram

The UML diagram below represents the corresponding use case (and their actors):

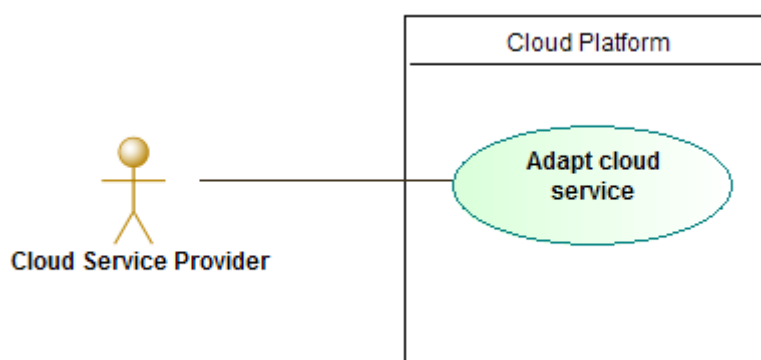


Figure 20 – implement CLARUS module for the cloud Use Case Diagram

1.1.1.48 Use cases

Adapt cloud service (UC-DEV-05)

UC-DEV-05	<i>Adapt cloud service</i>
Actors	Cloud service provider
Description	The cloud service provider wants to modify implementation of a cloud service in order to enhance security and trust capabilities
Pre-conditions	<i>None</i>
Main scenario	<i>Adapt cloud service</i>
Step 1	The cloud service provider gets the CLARUS development manual for CSP corresponding to the version of CLARUS library(ies) that is/will be installed on the cloud platform
Step 2	The cloud service provider gets the CLARUS library(ies), and installs it on the development platform
Step 3	The cloud service provider modifies the implementation of the cloud service by following the CLARUS development manual for CSP and using the CLARUS library(ies)
Step 4	The cloud service provider deploys the new implementation of the cloud service with the CLARUS library(ies) on the cloud platform
Post-conditions	
<i>[Post-condition 1]</i>	A security-enhanced and trust-enhanced version of the cloud service has been deployed

4 Refined requirements

4.1 Reminders

The aim of this section is to describe all the identified requirements of the different functions to be performed by CLARUS software. This gathering of requirements is inherited from the user requirements previously stated in the use cases section.

These requirements will provide a baseline for the subsequent steps of :

- Design and implementation of the CLARUS software solution (WP5),
- Evaluation, Testing and Validation of the solution (WP6)

Defining these requirements will aim to specify :

- Which functionalities should be supported in order to have a complete description of what CLARUS software is supposed to do and thus try to cover all the facets of its potential use (i.e functional requirements). We will base in particular on the different perspectives outlined in the use cases definition section (Section 0).
- Which are the expected outcome in terms of integrity, reliability, quality of service, security needs, legal constraints...

The review of the requirements is therefore organized in two main sections, namely: functional and non functional requirements. Each contains subsections, related to the issues identified.

4.1.1 Naming Scheme

In a standardisation purpose, the requirements will be assigned with an unique identifier defined by the following nomenclature :

<Requirement ID>= REQ-<Category code>-<Progressive number>
--

Figure 21 - Requirement Naming Scheme

Such a nomenclature will ensure the subsequent requirement traceability.

4.1.2 Priorities

“Requirements are organized and classified in a priority hierarchy in order to facilitate later adjustments to any limitations in their implementation that may be due to time and budget constraints” [A1].

Using the following classification of four categories allows us to propose a priority level organization in descending order :

- Must have [MH]: the project will be seriously impacted if this requirement is not met.
- Should have [SH]: a requirement that, if it is not met, seriously impacts the project, but it can be delivered on a different timescale than the must-haves.
- Could have [CH]: optional requirements which can improve the project outcome and the value of its results, but are not essential to the main delivery (not too many Could-have requirements should be dropped in the design).
- Would like [WL]: truly optional features which can be delivered if there is sufficient effort available within a task, or if they can be found elsewhere and integrated easily.

The priorities assigned to each requirement reflect the current state of our analysis of the CLARUS solution, and they might be subject to modification in the next version of the requirement specification according to constraints, risks or technical limitations that may emerge during the project development.

4.1.3 Description

The template table below will be used for requirements description :

Req. No.	Description	Priorities	Comment
<Requirement ID> REQ-<Category code>-<Nb>		<MH SH CH WL>	Ref : <Use Case Id > or <comment>

4.1.4 Categories code

We refer here to two different conditions:

1. If we focus on the description of the features required, defining CLARUS system behaviour and general functionality we consider the category of 'functional requirements'.
2. If we focus on term of constraints, targets or control mechanisms, we refer to the 'non functional requirements' category.

The different sub-classes of requirements for each of these two main categories are :

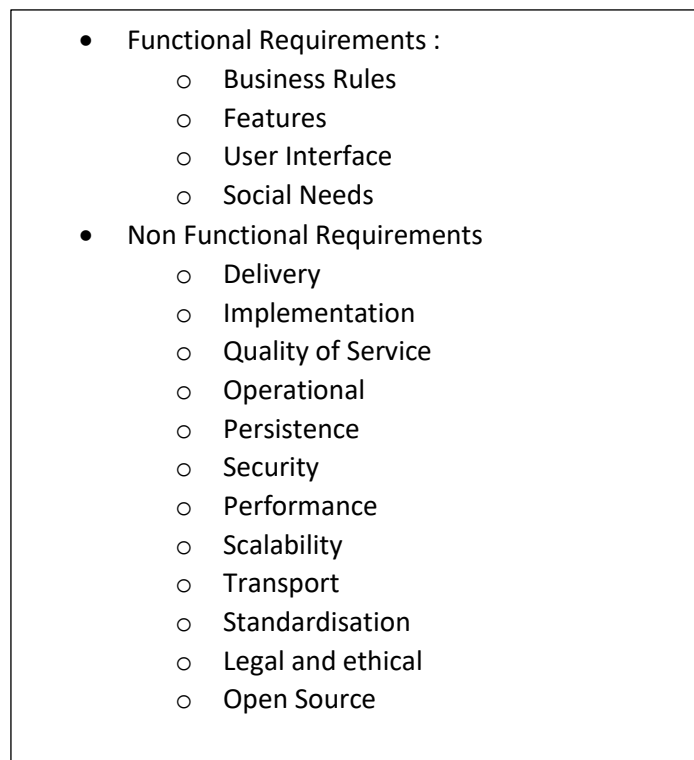


Figure 22 - Classification of Requirements

4.2 Functional Requirements

Functional requirements are the functions and features that the user needs for the system to work.

In this part, we address 4 main topics (Section 4.1.4), namely : business rules, features, user interface and social needs.

4.2.1 Business Rules

This set of requirements is linked to the category code : <F_BRU>.

We set out here the business requirements to enable the implementation and compliance with a business rule. These requirements correspond to high level needs of a business organization. They should describe what the CLARUS system should support. It is by their fulfilment that the organization achieves its objectives.

Req. No.	Description	Priorities	Comment
REQ-F_BRU-1.1	The CLARUS solution must help cloud customers increasing their control over their (critical and sensitive) outsourced data.	MH	Ref: UC-EUP-*
REQ-F_BRU-1.2	The CLARUS solution must offer security and privacy-enhanced mechanisms, which should be monitorable by the security manager of the customer's organization.	MH	
REQ-F_BRU-1.3	The CLARUS solution must not hinder the normal operation of the client applications on premises and/or the services on the cloud.	MH	Ref: UC-EUP-*
REQ-F_BRU-1.4	At any moment in the data securisation workflow, CLARUS utilities must not apply some unprotection mechanism that may lead to disclosure or leakage of sensitive or confidential data to the CSP(s).	MH	Ref: UC-EUP-*
REQ-F_BRU-1.5.1	The CLARUS solution implemented must offer an intrusion prevention and detection system.	MH	Ref: UC-MON-02, UC-MON-07, UC-MON-08, UC-MON-09
REQ-F_BRU-1.5.2	The intrusion detection system offered by CLARUS must provide dedicated countermeasures to manage any episode of intrusion and vulnerability on the client device, the CLARUS proxy, or the cloud	SH	Ref: UC-MON-02, UC-MON-07, UC-MON-08, UC-MON-09

	storage platform		
REQ-F_BRU-1.6	Any intrusion detection should be treated in an 'Attack Tolerant System' perspective allowing applications to continue to operate.	SH	Ref: UC-MON-07
REQ-F_BRU-1.7	The implementation of CLARUS must allow some of its users (e.g. security manager) to specify the level of security they want to apply.	MH	
REQ-F_BRU-1.8	The architecture/design of the CLARUS solution must allow the communication between several CLARUS proxies, and their interoperability.	MH	Ref: UC-SEC-13, UC-SEC-14
REQ-F_BRU-1.9.1	The CLARUS solution must support generic protocols and generic data formats.	MH	Ref: UC-SEC-02
REQ-F_BRU-1.9.2	The CLARUS solution must implement open standards in order to ensure interoperability and collaboration between several Cloud Service Providers.		
REQ-F_BRU-1.10	The CLARUS solution should not induce a loss of data utility for its end-users.	CH	
REQ-NF_BRU-1.11	The CLARUS solution should not compromise the following benefits offered by cloud computing : cost savings and ubiquitous access.	SH	
REQ-NF_BRU-1.12	The CLARUS solution should be able to partially preserve the analytical utility of the user data for the CSP so that it could still (partially) mine user data	SH	This requirement may be imposed by the CSP to collaborate or to offer its services (for free).

4.2.2 Features

This set of requirements is linked to the category code : <F_FEA>.

1.1.1.49 Secure data storage

Req. No.	Description	Priorities	Comment
----------	-------------	------------	---------

REQ-F_FEA-1.1	Only the users authorized by the CLARUS system are allowed to run a data storage operation to a CSP.	MH	Ref: UC-EUP-01, UC-EUP-06
REQ-F_FEA-1.2	The CLARUS proxy must ensure the protection of the datasets to be stored in the cloud.	MH	Ref: UC-EUP-01
REQ-F_FEA-1.3	The operations performed on the data by the CLARUS proxy must not hinder the interaction of the user with the client application and/or the cloud service (transparency).	MH	Ref : UC-EUP-01, UC-EUP-06
REQ-F_FEA-1.4	After data storage, success/failure notification must be identical with or without CLARUS	MH	Ref : UC-EUP-01, UC-MON-05
REQ-F_FEA-1.5	The CLARUS proxy should follow a distribution/replication methodology for outsourcing data in different CSPs.	SH	Ref : UC-EUP-01, UC-EUP-02
REQ-F_FEA-1.6	Only the users authorized by the CLARUS system are allowed to perform a retrieval operation on protected data.	MH	Ref : UC-EUP-02
REQ-F_FEA-1.7	Users other than the data owner may be allowed to perform a retrieval operation on protected data.	MH	Ref : UC-EUP-02
REQ-F_FEA-1.8	The CLARUS proxy must be able to retrieve the data previously protected and stored in the cloud by means of an identifier.	MH	Ref : UC-EUP-02
REQ-F_FEA-1.9	During a retrieval operation, the CLARUS system must define the appropriate mechanism to unprotect the data (depending on the protection technique applied at storage).	MH	Ref : UC-EUP-02
REQ-F_FEA-1.10	At any time of a retrieval operation, the data must not be disclosed unprotected to the CSP nor any other unauthorized party. Only the users authorized by the CLARUS system are allowed to access the unprotected (clear)	MH	Ref : UC-EUP-02

	data.		
REQ-F_FEA-1.11	Only the users authorized by the CLARUS system are allowed to perform a delete operation on protected data.	MH	Ref : UC-EUP-03, UC-EUP-07
REQ-F_FEA-1.12	The CLARUS proxy must be able to delete any data previously protected and stored in the cloud by means of an identifier.	MH	Ref : UC-EUP-03, UC-EUP-07
REQ-F_FEA-1.13	After data deletion, success/failure notification must be identical with or without CLARUS	MH	Ref : UC-EUP-03, UC-EUP-07, UC-MON-05

1.1.1.50 Secure data search

Req. No.	Description	Priorities	Comment
REQ-F_FEA-2.1	Only the users authorized by the CLARUS system are allowed to perform a search operation through the proxy on protected data.	MH	Ref: UC-EUP-04, UC-EUP-05
REQ-F_FEA-2.2	Users other than the data owner (e.g. data consumer) may be allowed to perform a search operation on protected data.	MH	Ref: UC-EUP-04, UC-EUP-05
REQ-F_FEA-2.3.1	The CLARUS solution must be able to process attribute-based search on protected data.	MH	Ref: UC-EUP-05
REQ-F_FEA-2.3.2	The CLARUS solution must be able to process “confidential queries” (i.e. queries containing sensitive information of the users).	MH	Ref: UC-EUP-05
REQ-F_FEA-2.4	The CLARUS solution must be able to unprotect the results of “confidential queries” (i.e. queries containing sensitive information of the users).	MH	Ref: UC-EUP-05
REQ-F_FEA-2.5	During a search operation, the content of the confidential query should no be disclosed to any party, except the authorized	CH	Ref: UC-EUP-05

	users and the CLARUS proxy.		
REQ-F_FEA-2.6	During a search operation, the CLARUS system must define the appropriate mechanism to unprotect the data (depending on the protection technique applied at storage).	MH	Ref: UC-EUP-04, UC-EUP-05
REQ-F_FEA-2.7	At any time of a search operation, the result must not be disclosed unprotected to the CSP nor any other unauthorized party. Only the users authorized by the CLARUS system are allowed to access the unprotected (clear) result of the search query.	MH	Ref: UC-EUP-04, UC-EUP-05
REQ-F_FEA-2.8	In the case of data attributes that have not been protected or that have been coarsened, it should be possible for users not authorized through CLARUS to perform search operations on these data attributes directly in the cloud, without the need of the CLARUS proxy.	SH	Ref: UC-EUP-05
REQ-F_FEA-2.9	In the case of data attributes that have not been protected or that have been coarsened, the CSP service should be able to provide the accurate or coarsened result of a search query on these data attributes.	SH	Ref: UC-EUP-05, UC-MON-05

1.1.1.51 Secure data update

Req. No.	Description	Priorities	Comment
REQ-F_FEA-3.1	Only the users authorized by the CLARUS system are allowed to perform an update operation on protected data.	MH	Ref: UC-EUP-06
REQ-F_FEA-3.2	The CLARUS solution must be able to process attribute-based updates on protected data, as long as the data have been protected with a technique supporting the update operation (e.g. data splitting).	MH	Ref: UC-EUP-06
REQ-F_FEA-3.4	During a search operation resulting from an	MH	Ref: UC-EUP-06

	update request, the content of the confidential search query must no be disclosed to any party, except the authorized users and the CLARUS proxy.		
REQ-F_FEA-3.5	The CLARUS solution should be able to process attribute-based updates on protected data, in such a way that the system is able, from “confidential search queries”, to update the corresponding data with the new protected value.	SH	Ref: UC-EUP-06
REQ-F_FEA-3.6	After data update, success/failure notification must be identical with or without CLARUS	MH	Ref : UC-EUP-06, UC-MON-05

1.1.1.52 Secure data computation

Req. No.	Description	Priorities	Comment
REQ-F_FEA-4.1	Only the users authorized by the CLARUS system are allowed to perform a compute operation through the proxy on protected data.	MH	Ref: UC-EUP-09
REQ-F_FEA-4.2	Users other than the data owner (e.g. data consumer) may be allowed to perform a compute operation on protected data.	MH	Ref: UC-EUP-09
REQ-F_FEA-4.3	The CLARUS solution must enable computation services hosted in the cloud to perform compute operations on protected data.	MH	Ref: UC-EUP-09
REQ-F_FEA-4.4	The CLARUS solution must manage the results of computation services on protected data previously stored in the cloud, in such a way that the system is able to provide the end user with the associated result in its unprotected format.	MH	Ref: UC-EUP-09
REQ-F_FEA-4.5	During a compute operation, the CLARUS system must define the appropriate mechanism to unprotect the data	MH	Ref: UC-EUP-09

	(depending on the protection technique applied at storage).		
REQ-F_FEA-4.6	At any time of a compute operation, the computation result must not be disclosed unprotected to the CSP nor any other unauthorized party. Only the users authorized by the CLARUS system are allowed to access the unprotected (clear) result of the computation.	MH	Ref: UC-EUP-09
REQ-F_FEA-4.7	In the case of data attributes that have not been protected or that have been coarsened, it should be possible for users not authorized through CLARUS to request computation services on these data attributes directly in the cloud, without the need of the CLARUS proxy.	SH	Ref: UC-EUP-10
REQ-F_FEA-4.8	In the case of data attributes that have not been protected or that have been coarsened, the CSP computation service should be able to provide the accurate or coarsened result of a computation over these data attributes.	SH	Ref: UC-EUP-10, UC-MON-05

1.1.1.53 Manage access control

Configure user repository

Req. No.	Description	Priorities	Comment
REQ-F_FEA-5.0	The CLARUS proxy must provide the IT team with the ability to configure the user repository from which the CLARUS proxy will retrieve the list of user identifiers.	MH	Ref : UC-ADM-03, UC-ADM-04
REQ-F_FEA-5.0.1	The CLARUS proxy must be able to accept a simple list of user identifiers as a “user repository” (e.g. text file, csv, etc.).	MH	
REQ-F_FEA-5.0.2	The CLARUS proxy must be able to accept a remote database as a “user repository”.	MH	

REQ-F_FEA-5.0.3	The CLARUS proxy must be able to accept a company directory as a “user repository”.	MH	
-----------------	---	----	--

Register/Unregister users

Req. No.	Description	Priorities	Comment
REQ-F_FEA-5.1	The CLARUS proxy must provide the security manager with the ability to register new users among the users found in the configured user repository.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.1.1	The CLARUS proxy must provide the security manager with the ability to search for users to be registered (by id and/or attributes) in the user repository.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.1.2	The CLARUS proxy should be able to access the identity service provided by the IT infrastructure, if any.	SH	Ref : UC-SEC-09
REQ-F_FEA-5.2	The CLARUS proxy must save the users that the security manager has registered (persistence).	MH	Ref : UC-SEC-09
REQ-F_FEA-5.5	The CLARUS proxy must offer the security manager a search facility to list all the CLARUS registered users.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.15	The CLARUS proxy must provide the security manager with the ability to allow users to access the outsourced datasets managed by the system.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.16	The CLARUS proxy must provide the security manager with a search facility to list the CLARUS users.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.25	The CLARUS proxy must provide the security manager with the ability to unregister users.	MH	Ref : UC-SEC-10
REQ-F_FEA-5.25.1	When a user is unregistered by the security manager, the CLARUS proxy must delete his/her profile.	MH	Ref : UC-SEC-06

Manage multiple proxies

Req. No.	Description	Priorities	Comment
REQ-F_FEA-5.10	The CLARUS proxy must provide the IT team and/or the security manager with the ability to define other CLARUS proxies to interoperate with.	MH	Ref : UC-SEC-13, UC-SEC-14

View/edit user authorizations

Req. No.	Description	Priorities	Comment
REQ-F_FEA-5.35	The CLARUS proxy must provide the security manager with the ability to edit a user profile (user authorizations on dataspace and outsourced datasets).	MH	Ref : UC-SEC-09
REQ-F_FEA-5.36	The CLARUS proxy must provide the security manager with the ability to define if a given user is allowed/not allowed to create new outsourced datasets.	MH	Ref : UC-SEC-09
REQ-F_FEA-5.41	The CLARUS proxy must provide the security manager with the possibility to view, for a given outsourced dataset the list of users who have authorizations on it.	MH	Ref : UC-SEC-10
REQ-F_FEA-5.42	The CLARUS proxy must provide the security manager with the possibility to view, for a given user and a given outsourced dataset, the authorizations the selected user has on this outsourced dataset (read, update, delete)	MH	Ref : UC-SEC-10
REQ-F_FEA-5.45	The CLARUS proxy must provide the security manager with the possibility to edit, in a secure way, the permissions (read, update, delete) of the users on an existing outsourced dataset.	MH	Ref : UC-SEC-10
REQ-F_FEA-5.45.1	The CLARUS proxy must provide the security manager with the possibility to update, for a	MH	Ref : UC-SEC-10

	given user and a given outsourced dataset: - The read permission of this user on this outsourced dataset (yes/no) - The update permission of this user on this outsourced dataset (yes/no) The delete permission of this user on this outsourced dataset (yes/no).		
REQ-F_FEA-5.49	The CLARUS proxy must provide the security manager with the possibility to list the registered users who do not have any authorization defined on a given outsourced dataset.	MH	Ref : UC-SEC-10
REQ-F_FEA-5.50	The CLARUS proxy must provide the security manager with the ability to add, for a given outsourced dataset, a new user in the list of authorized users and the corresponding permission (read, update, delete).	MH	Ref : UC-SEC-10

1.1.1.54 Manage security policy

Create a security policy for a dataspace

Req. No.	Description	Priorities	Comment
REQ-F_FEA-6.1	The CLARUS proxy must provide the security manager with the possibility to create a security policy for a dataspace.	MH	Ref : UC-SEC-01
REQ-F_FEA-6.5	The CLARUS proxy must provide the security manager with the ability to specify one or several data usage(s) for a given dataspace.	MH	Ref : UC-SEC-04
REQ-F_FEA-6.5.1	The data usages proposed by the CLARUS proxy to the security manager configuring a dataspace should be deduced from the use cases and scenarios (i.e. store, search, update, compute)	SH	Ref : UC-SEC-04
REQ-F_FEA-6.10	The CLARUS proxy must provide the security manager with the ability to specify, for a given dataspace, one or several protocols to access the data (e.g. Amazon S3, SQL, Web	MH	Ref : UC-SEC-02

	Service).		
REQ-F_FEA-6.10.1	The CLARUS proxy must provide the security manager with the ability, for a given protocol, to associate a dataspace with the endpoint of the cloud storage service.	MH	Ref : UC-SEC-02
REQ-F_FEA-6.10.2	The CLARUS proxy must provide the security manager with the ability to describe an endpoint according to the protocol, e.g.: - SQL protocol: the JDBC url (host, port, database), - Web service protocol: the WSDL url - Amazon S3 protocol: the access url (REST) - etc.	MH	Ref : UC-SEC-02
REQ-F_FEA-6.10.3	The CLARUS proxy should provide a facility for the security manager to test the connection to the protocol endpoint specified (result: connection ok / connection failed).	SH	Ref : UC-SEC-02
REQ-F_FEA-6.10.4	The CLARUS proxy must provide the security manager with the possibility to use additional modules for defining protocols not natively supported by CLARUS.	MH	Ref : UC-ADM-08
REQ-F_FEA-6.11	The CLARUS proxy should control that the protocol endpoint(s) associated to the dataspace belong to the CSP(s) configured during CLARUS installation.	SH	Ref : UC-SEC-02
REQ-F_FEA-6.12	In case of data splitting, the CLARUS proxy must control that more than one protocol endpoint are specified.	MH	Ref : UC-SEC-02
REQ-F_FEA-6.14	The CLARUS proxy must provide the security manager with the possibility to qualify the data attributes to be protected.	MH	Ref : UC-SEC-03
REQ-F_FEA-6.14.1	The CLARUS proxy could be able to visualize the structure of a database schema (given the connection url and credentials) and display it to the security manager for him/her to select data attributes to be protected.	CH	Ref : UC-SEC-03
REQ-F_FEA-6.15	The CLARUS proxy must provide the security	MH	Ref : UC-SEC-06

	manager with the ability to configure, for a given dataspace, the security rules that are the most appropriate.		
REQ-F_FEA-6.15.1	The CLARUS proxy must present to the security manager only those protection modules that are compatible with the data usage.	MH	Ref : UC-SEC-06,

1.1.1.55 Manage monitoring

Req. No.	Description	Priorities	Comment
REQ-F_FEA-7.1	The CLARUS proxy must provide the Security Manager and the IT team with the ability to monitor CLARUS usage and retrieve relevant metrics via additional relevant Monitoring and Attack management tools.	MH	Ref : UC-MON-01
REQ-F_FEA-7.2	The Monitoring tools and Attack management, additional to CLARUS, must contain an Intrusion Prevention System based upon a list of security incidents allowing definition and of dedicated counter-measures.	MH	Ref : UC-MON-08
REQ-F_FEA-7.3	The Monitoring tools and Attack management, additional to CLARUS, must contain an Intrusion Detection System based upon a list of metrics and indicators monitored allowing definition of rules to detect incidents.	MH	Ref : UC-MON-01, UC-MON-02, UC-MON-09
REQ-F_FEA-7.4	The Monitoring tools and Attack management, additional to CLARUS, must provide the authorized users with a web-based report which synthesizes all the monitoring information and relevant metrics retrieved, as well as intrusion detected and counter-measures associated in case of system attack.	MH	Ref : UC-MON-01, UC-MON-02, UC-MON-09
REQ-F_FEA-7.5	The Monitoring tools and Attack management, additional to CLARUS, must provide the Security Manager with the ability to get information and generates a	MH	Ref : UC-MON-06

	synthesizing web-based report coming from distributed CLARUS proxies.		
REQ-F_FEA-7.6	The Monitoring tools and Attack management, additional to CLARUS, must provide the Security Manager with the ability to be notified in real-time of an intrusion in the system.	MH	Ref : UC-MON-02, UC-MON-09
REQ-F_FEA-7.7	The Monitoring tools and Attack management, additional to CLARUS proxy(ies), must provide the authorized users with the ability to manually activate dedicated counter-measure(s) depending the attack type detected.	MH	Ref : UC-MON-02, UC-MON-09
REQ-F_FEA-7.8	The Monitoring tools and Attack management, additional to CLARUS proxy(ies), should provide the ability to trigger automated counter-measure(s) process by an attack detection.	SH	Ref : UC-MON-02, UC-MON-09
REQ-F_FEA-7.9	Functionality and performance of the Monitoring tools and Attack management, additional to CLARUS proxy(ies), should be tested in a dedicated integration environment.	SH	Ref : UC-MON-04
REQ-F_FEA-7.10	The Monitoring tools and Attack management, additional to CLARUS proxy(ies), must include the means to create and restore backup states of one or several CLARUS proxy(ies) in case of failure of the latter.	MH	Ref : UC-MON-03, UC-MON-06, UC-ADM07

1.1.1.56 Manage installation/configuration

Req. No.	Description	Priorities	Comment
REQ-F_FEA-8.1	It must be possible for an IT administrator to configure : - the parameters of the IT authentication service (if any) that CLARUS will use to authenticate the users (e.g. company SSO).	MH	Ref : UC-ADM-03, UC-ADM-05

	- an embedded authentication mechanism (if any) that CLARUS will use to authenticate the users.		
REQ-NF_FEA-8.2	It must be possible for an IT administrator to configure the cloud storage endpoints (URLs) and credentials of one (or more) cloud storage service(s) towards which CLARUS will direct the end-users.	MH	Ref : UC-ADM-03, UC-ADM-06
REQ-F_FEA-8.3	It must be possible for an IT administrator to add a module developed using the CLARUS development API.	MH	Ref : UC-IMPL-01
REQ-F_FEA-8.4	It must be possible for an IT administrator to configure the failover mode (activated/deactivated).	MH	Ref : UC-ADM-07
REQ-F_FEA-8.5	It must be possible for an IT administrator to configure the parameters specifying that a CLARUS proxy is part for the same failover cluster as other CLARUS proxies.	MH	Ref : UC-ADM-07

4.2.3 User Interface

This set of requirements is linked to the category code : <F_USI>.

1.1.1.57 General

Req. No.	Description	Priorities	Comment
REQ-F_USI-1.1	The CLARUS solution must provide user interfaces (either web-based interfaces or command line interfaces) for : - User authorization management - Security policy management - Administration - Monitoring and attack management - Protected data viewing	MH	Ref: UC-EUP-*

1.1.1.58 User authorization management

Req. No.	Description	Priorities	Comment
----------	-------------	------------	---------

REQ-F_USI-2.1	There must be a user interface for the security manager to connect to the CLARUS proxy.	MH	
REQ-F_USI-2.2	The CLARUS proxy must provide the security manager with an interface to manage the CLARUS users, namely the “user authorization management” interface.	MH	Ref: UC-SEC-09
REQ-F_USI-2.2.1	The user management interface of the CLARUS proxy should allow the security manager to register/unregister users.	SH	Ref: UC-SEC-09, UC-SEC-10
REQ-F_USI-2.2.2	The user management interface of the CLARUS proxy should allow the security manager to search for a user by attributes (e.g. login, mail address, name).	SH	Ref: UC-SEC-09
REQ-F_USI-2.5	The user management interface of the CLARUS proxy should allow the security manager to view/edit the user profile of a given user.	SH	Ref: UC-SEC-11, UC-SEC-12
REQ-F_USI-2.6	The user management interface of the CLARUS proxy should allow the security manager to view/edit user authorizations.	SH	Ref: UC-SEC-11, UC-SEC-12
REQ-F_USI-2.6.1	The user management interface of the CLARUS proxy should allow the security manager to display the list of outsourced datasets stored through the system.	SH	Ref: UC-SEC-12
REQ-F_USI-2.6.2	The user management interface of the CLARUS proxy should allow the security manager to select an outsourced dataset.	SH	Ref: UC-SEC-12
REQ-F_USI-2.6.3	The user management interface of the CLARUS proxy should allow the security manager to display the list of users which have authorizations on a selected outsourced dataset.	SH	Ref: UC-SEC-12
REQ-F_USI-2.6.4	The user management interface of the CLARUS proxy should allow the security manager to select a user which have	SH	Ref: UC-SEC-12

	authorizations on a given outsourced dataset.		
REQ-F_USI-2.6.5	The user management interface of the CLARUS proxy should allow the security manager to view/modify the authorizations a selected user has on a given dataset (read, update, delete).	SH	Ref: UC-SEC-12

1.1.1.59 Security policy management

Req. No.	Description	Priorities	Comment
REQ-F_USI-3.1	The CLARUS proxy must provide the security manager with an interface to manage the security policies, namely the “security policy management” interface.	MH	
REQ-F_USI-3.2	The security policy management interface of the CLARUS proxy should allow the security manager to create a security policy for a dataspace.	SH	
REQ-F_USI-3.3	The security policy management interface of the CLARUS proxy should allow the security manager to fill in the name of the dataspace to be created.	SH	
REQ-F_USI-3.4	The security policy management interface of the CLARUS proxy should allow the security manager to select among the list of available protocols. (e.g. SQL protocol, Web service protocol, Amazon S3 protocol, etc.)	SH	
REQ-F_USI-3.5	The security policy management interface of the CLARUS proxy should allow the security manager to configure the endpoint for each selected protocol (e.g. a file URI scheme, a JDBC url (host, port, database), a WSDL url, etc.)	SH	
REQ-F_USI-3.6.1	The security policy management interface of the CLARUS proxy should allow the security manager to display the list of attributes that	SH	Ref: UC-SEC-03

	could be protected (i.e. protocol attributes, attributes of file types, database attributes and columns to protect given a database schema)		
REQ-F_USI-3.6.2	The security policy management interface of the CLARUS proxy should allow the security manager to select the attributes to be protected	SH	Ref: UC-SEC-03
REQ-F_USI-3.6.3	The security policy management interface of the CLARUS proxy should allow the security manager to save the attributes to be protected.	SH	Ref: UC-SEC-03
REQ-F_USI-3.7.1	The security policy management interface of the CLARUS proxy should allow the security manager to display the list of possible data usages	SH	Ref: UC-SEC-04
REQ-F_USI-3.7.2	The security policy management interface of the CLARUS proxy should allow the security manager to select, among the usage list, the usage(s) of the outsourced datasets within the dataspace	SH	Ref: UC-SEC-04
REQ-F_USI-3.9.1	The security policy management interface of the CLARUS proxy should allow the security manager to display the list of security techniques that are the most appropriate according to the dataspace configuration that has been set	SH	Ref: UC-SEC-05
REQ-F_USI-3.9.2	The security policy management interface of the CLARUS proxy should allow the security manager to select, among the list of possible options, the security technique to be applied	SH	Ref: UC-SEC-05
REQ-F_USI-3.10	The security policy management interface of the CLARUS proxy should allow the security manager to create the dataspace with all the configuration parameters selected	SH	Ref: UC-SEC-01
REQ-F_USI-3.11	The security policy management interface of the CLARUS proxy should allow the security	SH	Ref: UC-SEC-01

	manager to configure a dataspace.		
REQ-F_USI-3.12	The security policy management interface of the CLARUS proxy should allow the security manager to choose the dataspace to be configured among the list of existing dataspace.	SH	Ref: UC-SEC-01

1.1.1.60 Monitoring and attack management interface

Req. No.	Description	Priorities	Comment
REQ-F_USI-4.1	The CLARUS interface must include a Monitoring and Attack Management Dashboard (MAMD) that will be accessible by the Security Manager and the IT Team.	MH	Ref: UC-MON-01
REQ-F_USI-4.2	The MAMD must include a list of metrics that can be measured by the monitoring tools and options to show (description, current value), select, and unselect specific metrics.	MH	Ref: UC-MON-01, UC-MON-07
REQ-F_USI-4.3	The MAMD must include a list of counter-measures and the following options: show description, select, unselect, activate, and deactivate.	MH	Ref: UC-MON-07, UC-MON-08, UC-MON-09
REQ-F_USI-4.4	The MAMD must include a list of possible attacks, an option to show the description of each attack and the suggested counter-measures to mitigate it, as well as the possibility of configuring (i.e., adding and removing) counter-measures to be automatically applied for a specific attack.	MH	Ref : UC-MON-08, UC-MON-09
REQ-F_USI-4.5	The MAMD must include the possibility of selecting and retrieving the value of a list of metrics: - From an individual CLARUS proxy, - Correlated from a list of CLARUS proxies.	MH	Ref: UC-MON-01, UC-MON-06
REQ-F_USI-4.6	The MAMD must include the possibility of	MH	Ref : UC-MON-08, UC-MON-09

	activating one or several counter-measures.		
REQ-F_USI-4.7	The MAMD should include the possibility of configuring the receiver of the attack alarms and reports (in case there are more than one Security Manager or that any member from the IT Team should be informed).	SH	Ref : UC-MON-08, UC-MON-09
REQ-F_USI-4.8	The MAMD should have the possibility of scheduling automatic report forwarding.	SH	Ref : UC-MON-02, UC-MON-09
REQ-F_USI-4.9	The MAMD must include the possibility of selecting and downloading existing reports.	MH	Ref : UC-MON-02, UC-MON-09
REQ-F_USI-4.10	The MAMD must include the means to create and restore backup states.	MH	Ref : UC-MON-03

4.2.4 Social needs

This set of requirements is linked to the category code : <F_SOC>.

Req. No.	Description	Priorities	Comment
REQ-F_SOC-1.1	The CLARUS solution, adopted by a customer who outsources his/her data to a CSP, must improve the trust of the customers in data storage with a cloud computing solution.	MH	
REQ-F_SOC-1.2	The architecture of CLARUS solution should be designed in a service-oriented architecture, with friendly interfaces to run the security mechanisms.	SH	Ref: UC-EUP-01
REQ-F_SOC-1.3	The usability of the CLARUS interfaces could be improved thanks to graphical user interfaces.	CH	
REQ-F_SOC-1.4	The usability of CLARUS interfaces could be improved thanks to comprehensive user manuals.	CH	
REQ-F_SOC-1.5	The usability of CLARUS interfaces could be improved thanks to clear and non ambiguous instructions as well as	CH	

	informative error messages.		
REQ-F_SOC-1.6	The usability of CLARUS interfaces could be improved thanks to help facilities.	WL	
REQ-F_SOC-1.7	The instructions displayed to end-users by CLARUS user interfaces must be submitted in the same language as used for the CLARUS agreement document [A2].	MH	
REQ-F_SOC-1.8	The usability of CLARUS interfaces could be improved thanks to multi-lingual features, providing the end users with the choice to display the instructions in a given language.	WL	

4.3 Non Functional Requirements

We call non-functional requirements the characteristics of the system that are important to the users even though they may not realize it. As the name suggests these requirements do not deal with a functionality of the system, but rather place constraints on how the system should operate.

This set of requirements allows to express some performance characteristics of CLARUS system, sometimes measurable.

We will address in this part the following main topics already set, namely :

- Delivery
- Implementation
- Quality of Service
- Operational
- Persistence
- Security
- Performance
- ScalabilityTransportStandardisation
- Legal and ethical
- Open Source

4.3.1 Delivery

This set of requirements is linked to the category code : <NF_DLV>.

1.1.1.61 Delivery requirements

Req. No.	Description	Priorities	Comment
REQ-NF_DLV-1.1	An installation package should be available for an IT administrator to install easily the	SH	Ref : UC-ADM-01

	CLARUS proxy on a given operating system (e.g. Unix/Linux).		
REQ-NF_DLV-1.2	CLARUS solution assumes a collaborative agreement by the CLARUS adopting CSPs, which must be willing to comply the need to install a CLARUS library in the cloud platform.	MH	Ref : UC-ADM-09
REQ-NF_DLV-1.3	It must be possible to upgrade CLARUS to a newer version, containing additional/ updated modules.	MH	Ref : UC-ADM-01, UC-ADM-02, UC-ADM-10
REQ-NF_DLV-1.4	The CLARUS solution must be delivered with an installation guide (or CSP installation guide).	MH	Ref : UC-ADM-01, UC-ADM-09
REQ-NF_DLV-1.5	The CLARUS solution must be delivered with a configuration guide.	MH	Ref : UC-ADM-03
REQ-NF_DLV-1.6	The CLARUS solution must be delivered with a development API.	MH	Ref : UC-IMPL-01
REQ-NF_DLV-1.7	The development API for CLARUS modules must be delivered with a development guide.	MH	Ref : UC-IMPL-01
REQ-NF_DLV-1.8	The CLARUS solution must be delivered with a monitoring and intrusion management solution.	MH	Ref : REQ-F_FEA-7.1, UC-MON-01

1.1.1.62 Configuration Management

Req. No.	Description	Priorities	Comment
REQ-NF_DLV-1.9	A configuration management methodology could be applied on the CLARUS software changes to handle the different releases of the modules/packages developed.	CH	Ref : UC-ADM-02
REQ-NF_DLV-1.10	A configuration management methodology could be applied on the documentation delivered with the CLARUS solution (installation guide, configuration guide, etc.) to handle the different releases of each document.	CH	Ref : UC-ADM-01, UC-ADM-03

REQ-NF_DLV-1.11	A configuration management methodology could be applied on software changes of the CLARUS development API to handle the different releases of the CLARUS development API.	CH	Ref : UC-IMPL-01
REQ-NF_DLV-1.12	A configuration management methodology could be applied on the documentation delivered with the CLARUS development API (development manual) to handle the different releases of the document.	CH	Ref : UC-IMPL-01
REQ-NF_DLV-1.13	A configuration management methodology could be applied on software changes of the CLARUS Monitoring and Attack management tools, to handle the different releases.	CH	Ref : UC-MON-01
REQ-NF_DLV-1.14	A configuration management methodology could be applied on the documentation delivered with the CLARUS Monitoring and Attack management tools to handle the different releases of the associated documents.	CH	Ref :C-MON-01

4.3.2 Implementation

This set of requirements is linked to the category code : <NF_IMP>.

Req. No.	Description	Priorities	Comment
REQ-NF_IMP-1.1	The CLARUS solution implementation must allow multi OS (e.g. Windows, Linux...) compatibility.	MH	
REQ-NF_IMP-1.2	Storage service must be configured to allow access only from the end user premises.	MH	Ref : UC-EUP-01
REQ-NF_IMP-1.3	For storage application case, the end user of CLARUS solution must have a user account with sufficient privileges to access the CSP data storage space.	MH	Ref : UC-EUP-01, UC-EUP-02, UC-EUP-03
REQ-NF_IMP-1.4	The CLARUS system must be able to identify the dataset to which a data belongs by	MH	Ref : UC-EUP-01

	means of an identifier.		
REQ-NF_IMP-1.5	The CLARUS system must support different data types as structured data (e.g. relational database) as well as unstructured (e.g. binary file, image, text file).	MH	Ref : UC-EUP-01, UC-EUP-02, UC-EUP-03
REQ-NF_IMP-1.6	CLARUS solution assumes a collaborative agreement by the CLARUS adopting CSPs, which must be willing to comply the need to adapt the implementation of cloud services if required (in order to integrate CLARUS features required to enhance security and trust in storage or some computational cloud services).	MH	Ref : UC-IMPL-02

4.3.3 Development

This set of requirements is linked to the category code : <NF_DEV>.

These requirements are dedicated to developers who are willing to extend CLARUS and develop specific plugins.

1.1.1.63 Source code

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.1	The CLARUS source code should be accessible on a public code repository.	SH	Ref : UC-DEV-01

1.1.1.64 Configuration management

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.11	The different releases of the CLARUS source code (modules/packages) should be managed via a distributed version control system.	SH	Ref : UC-ADM-02
REQ-NF_DEV-1.12	The different versions of the CLARUS documentation (installation guide, configuration guide, etc.) could be managed according to a configuration management methodology.	CH	Ref : UC-ADM-01, UC-ADM-03

REQ-NF_DEV-1.13	The different releases of the CLARUS development API should be managed via a distributed version control system.	SH	
REQ-NF_DEV-1.14	The different versions of the CLARUS development guide could be managed according to a configuration management methodology.	CH	
REQ-NF_DEV-1.15	The different releases of the CLARUS Monitoring and Attack management tools could be managed via a software configuration management system.	CH	Ref : UC-MON-01
REQ-NF_DEV-1.16	The different releases of the CLARUS Monitoring and Attack management documentation could be managed according to a configuration management methodology.	CH	Ref :C-MON-01

1.1.1.65 Documentation

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.21	There must be a developer guide for the developers willing to contribute to the CLARUS open source solution.	MH	Ref : UC-DEV-01
REQ-NF_DEV-1.22	The CLARUS developer guide should describe step by step how to extend the CLARUS solution.	SH	Ref : UC-DEV-01
REQ-NF_DEV-1.23	The CLARUS development API should be well documented		Ref : UC-DEV-01
REQ-NF_DEV-1.24	The latest version of the documentation should be available on the CLARUS code repository and/or CLARUS website.	SH	Ref : UC-DEV-01
REQ-NF_DEV-1.25	There could be a coding style guide for the developers willing to contribute to the CLARUS open source solution.	CH	Ref : UC-DEV-01

1.1.1.66 Issues

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.31	Bug reports could be managed via an issue tracker system.	CH	Ref : UC-DEV-05

1.1.1.67 New feature requests

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.41	Enhancements and new feature requests could be managed via an issue tracker system.	CH	Ref : UC-DEV-05

1.1.1.68 Communication

Req. No.	Description	Priorities	Comment
REQ-NF_DEV-1.51	There should be a CLARUS open source community manager responsible for support and/or maintenance.	SH	Ref : UC-DEV-01
REQ-NF_DEV-1.52	The CLARUS website should contain information for the developers.	SH	
REQ-NF_DEV-1.53	There should be a communication medium (e.g. mailing list) to centralize all CLARUS development discussions.	SH	

4.3.4 Quality of Service

This set of requirements is linked to the category code : <NF_QOS>.

Req. No.	Description	Priorities	Comment
REQ-NF_QOS-1.1	Any end-user applications interfaced with CLARUS features, must maintain their ability to perform the required usual functions as a guaranteed reliability.	MH	
REQ-NF_QOS-1.2	CLARUS securing tool should be an “Attack Tolerant System” for the cloud, so that it continues to operate and deliver the	SH	Ref : UC-MON-07

	required services under the event of an attack.		
REQ-NF_QOS-1.3.1	CLARUS must not impair typical cloud functionality (including the functionality provided by high-level services such as data storage, management, retrieval, transformation, etc.)	MH	CLARUS must not imply loss of functionality, i.e. make it hard or impossible to perform certain operations on data
REQ-NF_QOS-1.3.2	The benefits of an ubiquitous access of data outsourced offered by a high availability service of a CSP, should be maintained after adopting the CLARUS solution.	SH	
REQ-NF_QOS-1.4	CLARUS must implement the appropriate techniques to ensure the reliability and availability of the service (e.g. provide a fail-over mode).	MH	To be validated through metrics e.g. failure rate, mean time between failures (example: shall have an availability of 95%)
REQ-NF_QOS-1.5	The CLARUS solution should be able to prevent the complete loss of services availability during attacks.	SH	I.e. graceful degradation of services

4.3.5 Operational

This set of requirements is linked to the category code : <NF_OPE>.

Req. No.	Description	Priorities	Comment
REQ-NF_OPE-1.1	Each application case of data outsourcing to the cloud (and data modification) could be complemented by data distribution/replication between distincts CSPs or in distincts location/data centers of an unique CSP, in a Disaster Recovery approach.	CH	Ref : UC-EUP-01, UC-EUP-03
REQ-NF_OPE-1.2	In case of failure of the main CSP, data integrity and availability could be ensured by the “Disaster Recovery” center designated by the CSP.	CH	Ref : UC-EUP-01, UC-EUP-02, UC-EUP-03
REQ-NF_OPE-1.3	The solution must be robust against local CLARUS failures.	MH	

4.3.6 Persistence

This set of requirements is linked to the category code : <NF_PER>.

The persistence set of requirements details operational and performance criteria relating to the storage of information backup of database and files system and other persistent storage mechanisms.

Req. No.	Description	Priorities	Comment
REQ-NF_PER-1.1	The business continuity linked to the persistence and the recovering of the data outsourced, after protection by the CLARUS solution, will be ensured by the CSP in a Disaster Recovery plan.	SH	Ref : REQ-NF_OPE-1.1, REQ-NF_OPE-1.1
REQ-NF_PER-1.2	All the configuration components of a CLARUS proxy must be subject to a backup in case of outage of the CLARUS proxy (requiring a new CLARUS proxy reconfiguration or the initial CLARUS proxy restoration).	MH	Ref : REQ-F_FEA-7.10, UC-MON-03
REQ-NF_PER-1.3	The metadata stored by CLARUS must be stored as a backup.	MH	Either on premises or in the cloud (protected)
REQ-NF_PER-1.4	The backup of the metadata stored by CLARUS must be protected against the CSP and any other unauthorised party.	MH	

4.3.7 Security

This set of requirements is linked to the category code : <NF_SEC>.

The security set of requirements details both the information security (data access) and physical security (servers, applications and critical hardware access).

Req. No.	Description	Priorities	Comment
REQ-NF_SEC-1.1	The access permissions to CLARUS system may only be changed by the CLARUS security manager, or the like, of the organization concerned.		
REQ-NF_SEC-1.2	The CLARUS solution must ensure that data is protected from unauthorised access.	MH	

REQ-NF_SEC-1.3	The CLARUS solution should be able to isolate attacks and contain them.	SH	
REQ-NF_SEC-1.4	The CLARUS solution must be able to regenerate after an attack, and ensure the integrity of the system.	MH	i.e. get back to a previous state
REQ-NF_SEC-1.5	The CLARUS solution must offer a quality of data protection satisfying for the organization concerned, by the means of dedicated security mechanisms, on each type of data where privacy or confidentiality is a major concern.	MH	CLARUS must not imply a lower degree of security, e.g. in the use of function-preserving encryption methods.

4.3.8 Performance

This set of requirements is linked to the category code : <NF_PERF>.

The performance set of requirements can be linked with parameters such as response time and such measurable aspects of the system that govern overall speed and responsiveness.

Req. No.	Description	Priorities	Comment
REQ-NF_PERF-1.1	The securisation mechanisms introduced and embodied by CLARUS must not significantly degrade the response time compared to the normal behaviour of data processing.	MH	
REQ-NF_PERF-1.2	CLARUS should improve the performance of requests (e.g. search, operations) compared to naïve implementations of security-enabling solutions already available in some commercial products.	SH	Naïve implementations would be e.g. download the whole encrypted/ anonymised dataset from the CSP to the CLARUS proxy; perform the search operations on the proxy over clear data; update and re-encrypt / anonymise the results and re-upload them to the CSP
REQ-NF_PERF-1.3	CLARUS should improve the performance of requests (e.g. search, operations) compared to the same approach directly on the end user's computer.	SH	Without considering the case of not using the cloud at all (i.e. to run the whole application on the end user computer).

4.3.9 Scalability

This set of requirements is linked to the category code : <NF_SCA>.

Scalability is the ability of the system to handle an increased workload without significantly increasing the transaction processing time.

Req. No.	Description	Priorities	Comment
REQ-NF_SCA-1.1	The CLARUS solution must implement the appropriate techniques (distributed based system) to ensure the scalability of the system.	MH	e.g. replication, parallelization

4.3.10 Transport

This set of requirements is linked to the category code : <NF_TRSP>.

The transport set of requirements defines the constraints affecting the transmission of information between nodes.

Req. No.	Description	Priorities	Comment
REQ-NF_TRSP-1.1	The CLARUS solution must integrate and support secured communication protocols for the data management and transfers to the cloud storage environment.	MH	Ref : UC-SEC-11 e.g. HTTPS, SSL
EQ-NF_TRSP-1.2	The CLARUS solution must support the secured protocols used by cloud services with which CLARUS interacts for data processing.	MH	Ref : UC-SEC-11
REQ-NF_TRSP-1.3	The CLARUS solution must seamlessly integrate the network infrastructure, in order to not induce any significant latency in the requests processing.	SH	

4.3.11 Standardisation

This set of requirements is linked to the category code : <NF_STD>.

As previously mentioned, as part of WP2, this current document is part of a suite of related deliverable, in particular D2.5 [R2]. This latter, outlines standardisation requirements and provides a roadmap aiming to integrate the existing standards used, function of the different CLARUS fields of application. Consequently, we voluntarily approach this topic in the form of a generic requirement.

Please refer to the D2.5 for the detailed CLARUS requirements regarding standardisation.

Req. No.	Description	Priorities	Comment
REQ-NF_STD-1.1	The CLARUS solution must support most of the standards used in the different business domains on which CLARUS is deployed.	MH	

4.3.12 Legal and ethical

This set of requirements is linked to the category code : <NF_LEG>.

This set of legal and ethical requirements is further extended and developed in the D2.4 deliverable [R1], where data privacy and confidentiality concerns are detailed for the domains of the CLARUS application cases (i.e Geospatial publications and e-Health).

Req. No.	Description	Priorities	Comment
REQ-NF_LEG-1.1	Any access authentication controls to CLARUS proxy(s) (based on authorisations delivered to an end user on existing outsourced dataset, see UC-EUP-06), processing personal data to do so in a secure manner, respecting Article 17 Directive 95/46/EC. The data minimization principle and purpose limitation principle (as contained in Article 6 Directive 95/46/EC) should also be respected.	MH	Directive 95/46/EC : [R3] Ref : UC-SEC-09
REQ-NF_LEG-1.2	Any personal data related to the authenticated users, must be securely protected throughout the data lifecycle. As per Article 6 Directive 95/46/EC, any data that is not necessary for the personal data processing should be deleted in compliance with the limited retention and data minimisation principles.	MH	Directive 95/46/EC : [R3] Ref : UC-SEC-09
REQ-NF_LEG-1.3	Any personal data related to the authenticated users, must be securely protected throughout the data lifecycle : • Encryption should be employed throughout with the default state of data being unreadable if there is a data leak. • In respect of the data subject's rights,	MH	Directive 95/46/EC : [R3] Ref : UC-SEC-09

	any request to delete personal data should be respected. Any processing of personal data must be transparent so that the end user is aware of the nature of the processing and can exercises his/her data subject rights in an unencumbered fashion.		
REQ-NF_LEG-1.4	Any communications sent between the authenticated users and the CLARUS adopting CSP to be encrypted in order to prevent any unauthorized access and ensure the security of the data being shared. Systems should be designed to ensure that even where personal data are transmitted, any data elements which are not necessary to fulfil the purpose of the transmission are filtered out or removed.	MH	Directive 95/46/EC : [R3] Ref : UC-SEC-09
REQ-NF_LEG-1.5	The cloud storage of any personal (sensitive) data, processed by CLARUS solution must be compliant in respect of article 17 Directive 95/46/EC, namely that such storage must be secure.	MH	Directive 95/46/EC : [R3] Ref : UC-EUP-01
REQ-NF_LEG-1.6	CLARUS system must be designed so as to allow access to the transferred personal data only to the extent necessary for the role being performed and the individual performing it. Such access rights should be easily revocable.	MH	Directive 95/46/EC : [R3] Ref : UC-EUP-01, UC-SEC-04, UC-SEC-06, UC-SEC-09
REQ-NF_LEG-1.7	Ensure that additional legal and ethical requirements, set out in deliverable D2.4, are embedded in CLARUS solution.	MH	D2.4 deliverable [R1]

4.3.13 Open Source

This set of requirements is linked to the category code : <NF_OPS>.

Req. No.	Description	Priorities	Comment
REQ-NF_OPS-1.1	The CLARUS softwares implemented should be an open source solution.	SH	

5 Conclusion

This document contains several updates and enhancements compared to the first version of the CLARUS user and technical requirements. While keeping the same key assumptions about what the CLARUS solution is, this version is more in line with the intermediate results of the project. To summarize these results and their impact on the requirements, we can say that:

- the research on security and privacy (WP3) led us to distinguish between use cases where the data to be protected need to be uploaded once *as a whole dataset*, and others where the data could be protected and stored *record by record*. Furthermore, the research works helped us refine the definition of CLARUS security policies, introducing among others the notion of *attribute type*,
- the main result from the architecture design (WP4), as far as a revamping of requirements is concerned, is the *data routing* option for managing several proxies. As no import nor export of users is needed in this case, the inter-proxy user management described in the initial version gets significantly simpler. Specific uses cases have been added though, describing this new (preferred) way to register/unregister external proxies,
- the first results from the implementation phase (WP5) led us to reconsider the description of user interfaces, in order not to anticipate the choice of developing web interfaces rather than command line interfaces (command-line interfaces may be preferable in the case of administration tools dedicated to super-users only),
- the evaluation, testing and validation phase (WP6) did not lead us to reconsider the non-functional requirements.

In addition, recommendations from the Interim Review have been taken into account, leading to an extension of the requirements in order to represent the developer's requirements.

We believe that this version of the document will further ease the understanding of the user and technical requirements for CLARUS, and provide the reader with a comprehensive view of the challenges related to data protection in the cloud.